

From: <https://chatgpt.com/c/6a7f33eb-07f4-83ee-8229-383f952dcf93>

you asked: spring boo

message time: 2026-08-14 20:59:23

spring boot interviews latest for 2 year experience with questions and answers asked in the MANGOS companies so on that questions based i can easily crack the interview rounds in top product based companies or MANGOS comanies, give me all the questions with answers after going through websites, online published quetions or any other references of videos or youtubes then give all in proper format and explanation of concepts. X10THINK

chatgpt response: Absolutely

Absolutely. I treated “MANGOS” as **FAANG/MANGA-style top product companies** and built this around what a **2-year Java + Spring Boot backend engineer** is realistically expected to handle.

I also checked recent interview reports and published question banks rather than giving you a generic Spring Boot list. Recent reports for **Mastercard, Target, Cisco, Morgan Stanley, Apptio, Infosys, Accenture, HCL and others** repeatedly show the same pattern: **Core Java + Spring/Spring Boot + REST + JPA/Hibernate + SQL + multithreading + microservices + project deep-dive + DSA**, with product companies adding more scenario/LLD depth. ([GeeksforGeeks](#))

The latest Spring Boot question collections also emphasize `@SpringBootApplication`, auto-configuration, starters, component scanning, configuration, REST, Actuator and production concerns. ([GeeksforGeeks](#))

□ 2-Year Java + Spring Boot Product-Company Interview Master Guide

First: What you actually need to crack

For ~2 years experience, don't prepare only:

“What is Spring Boot?”

A good product-company interviewer will often go:

What is Spring Boot? ↓ How does auto-configuration work? ↓ What happens internally when your application starts? ↓ What if two beans have the same type? ↓ How does @Transactional work? ↓ What happens if an exception occurs? ↓ How would you debug a slow API? ↓ How would you scale this API?

That **follow-up chain** is what you need to prepare for.

□ Priority order

For a 2-year Java backend candidate, I would prepare in this order:

Priority	Topic	Importance
□ 1	Core Java	★ ★ ★ ★ ★
□ 2	Spring Core / IoC / DI	★ ★ ★ ★ ★
□ 3	Spring Boot	★ ★ ★ ★ ★
□ 4	REST APIs	★ ★ ★ ★ ★
□ 5	JPA/Hibernate	★ ★ ★ ★ ★
□ 6	SQL	★ ★ ★ ★ ★
□ 7	Microservices	★ ★ ★ ★ ★
□ 8	Multithreading	★ ★ ★ ★
□ 9	DSA	★ ★ ★ ★
□ 10	Spring Security/JWT	★ ★ ★ ★
□ 11	Kafka/Redis	★ ★ ★
□ 12	Docker/Cloud	★ ★ ★
□ 13	System/LLD	★ ★ ★
□ 14	Design patterns	★ ★ ★

Actual interview reports support this combination. For example, Target reported HashMap, DI, server configuration, database connectivity, load balancing, circuit breakers, Docker and environment configuration; Apptio reported Spring transactions, `@Autowired`, `@Qualifier`, `@Query`, JPA relationships, HashMap, threads and design patterns. ([GeeksforGeeks](#))

PART 1 — SPRING CORE

Q1. What is Spring?

Strong interview answer

Spring is a Java framework that simplifies enterprise application development by providing features such as:

- IoC/Dependency Injection
- AOP
- transaction management
- web development

- database integration
- security integration
- testing support

The central idea is that **Spring manages application objects, called beans, instead of the application creating and managing all dependencies manually.**

Example

Without Spring:

```
java
PaymentService paymentService =
    new PaymentService(new PaymentRepository());
```

With Spring:

```
java
@Service
public class PaymentService {

    private final PaymentRepository repository;

    public PaymentService(PaymentRepository repository) {
        this.repository = repository;
    }

}
```

Spring creates and injects `PaymentRepository`.

Follow-up

Why is this useful?

Because it gives us:

- loose coupling
- easier testing
- easier replacement of implementations
- centralized configuration
- better maintainability

Q2. What is IoC?

IoC = Inversion of Control.

Normally your application controls object creation:

```
text
Application
  ↓
new PaymentService()
  ↓
new PaymentRepository()
```

With Spring:

```
text
Spring Container
  ↓
creates Repository
  ↓
creates Service
  ↓
injects Repository into Service
```

The control of object creation has moved from your application code to the Spring container.

That's **Inversion of Control**.

Q3. What is Dependency Injection?

Dependency Injection is the mechanism through which Spring supplies an object's required dependencies.

Example:

```
java
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

}
```

`OrderService` depends on `PaymentService`.

Spring injects it.

Types of DI

1. Constructor injection
2. Setter injection
3. Field injection

Which one should you prefer?

Constructor injection.

Why?

- dependencies are explicit
- fields can be `final`
- easier unit testing
- object cannot exist without required dependencies
- avoids reflection-based field injection

Q4. `@Component` vs `@Service` vs `@Repository` vs `@Controller`

All are Spring-managed components, but communicate different responsibilities.

Annotation	Purpose
<code>@Component</code>	Generic Spring bean
<code>@Service</code>	Business/service layer
<code>@Repository</code>	Data-access layer
<code>@Controller</code>	MVC controller
<code>@RestController</code>	REST API controller

A strong answer:

@Service, @Repository, and @Controller are specialized stereotypes of @Component used to express architectural responsibility. @Repository additionally participates in Spring's persistence exception translation.

Q5. What is a Bean?

A Spring Bean is an object whose lifecycle is managed by the Spring IoC container.

Example:

```
java
@Service
public class UserService {
}
```

Spring discovers the class and creates/manages an instance.

You can also explicitly define one:

```
java
@Configuration
public class AppConfig {

    @Bean
    public ObjectMapper objectMapper() {
        return new ObjectMapper();
    }
}
```

Q6. What is ApplicationContext?

`ApplicationContext` is the central Spring container responsible for managing beans.

It handles things such as:

- bean creation
- dependency injection
- bean lifecycle
- configuration
- events
- resource loading

Think:

```
text
ApplicationContext
|
+--- Bean creation
+--- Dependency injection
+--- Lifecycle
```

```
+--- Configuration
+--- Events
```

Q7. What is `@Autowired`?

`@Autowired` tells Spring to resolve and inject a dependency.

Example:

```
java
@Autowired
private PaymentService paymentService;
```

However, for modern Spring applications, prefer:

```
java
private final PaymentService paymentService;

public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

Spring supports constructor injection without requiring `@Autowired` when there is a single constructor.

Q8. What happens if two beans have the same type?

Example:

```
java
@Service
class PayPalPaymentService implements PaymentService {}

@Service
class StripePaymentService implements PaymentService {}
```

Then:

```
java
public OrderService(PaymentService paymentService)
```

is ambiguous.

Spring can use:

`@Primary`

```
java
@Primary
@Service
class PayPalPaymentService implements PaymentService {
}
```

`@Qualifier`

```
java
public OrderService(
    @Qualifier("stripePaymentService")
    PaymentService paymentService) {
}
```

Spring's current documentation explicitly describes `@Primary` for resolving multiple autowire candidates. ([Home](#))

Q9. What are Spring bean scopes?

Important ones:

Singleton

Default.

One bean instance per Spring application context.

Prototype

A new instance is created whenever requested.

Request

One bean instance per HTTP request.

Session

One instance per HTTP session.

Spring's component scanning documentation confirms singleton as the default scope and supports explicit scopes such as prototype. ([Home](#))

Q10. What is bean lifecycle?

Simplified:

text

```
Bean definition
    ↓
Bean instantiated
    ↓
Dependencies injected
    ↓
@PostConstruct
    ↓
Bean ready
    ↓
Application shutdown
    ↓
@PreDestroy
```

Typical interview annotations:

java

```
@PostConstruct
public void init() {
}

@PreDestroy
public void cleanup() {
}
```

PART 2 — SPRING BOOT

Q11. Spring vs Spring Boot?

This is **extremely common**.

Spring

Spring is the framework/ecosystem.

Historically, applications often required considerable explicit configuration.

Spring Boot

Spring Boot simplifies Spring application development using:

- auto-configuration
- starter dependencies
- embedded servers
- opinionated defaults
- externalized configuration
- production features

Interview answer

Spring provides the underlying framework and programming model, while Spring Boot builds on Spring to reduce configuration and make it easier to create production-ready applications quickly.

Q12. What is Spring Boot?

Spring Boot is an opinionated framework built on top of Spring that helps create stand-alone, production-ready applications with minimal configuration.

Major features:

```
text
Spring Boot
├─ Auto Configuration
├─ Starter Dependencies
├─ Embedded Server
├─ Externalized Configuration
├─ Actuator
└─ Production-oriented features
```

Q13. What does `@SpringBootApplication` do?

This is one of the **highest-priority questions**.

```
java
@SpringBootApplication
public class Application {
}
```

Conceptually:

```
java
@SpringBootApplication
```

```
=
@Configuration
+
@EnableAutoConfiguration
+
@ComponentScan
```

`@Configuration`

Marks the class as configuration.

`@EnableAutoConfiguration`

Allows Spring Boot to configure beans based on classpath/configuration.

`@ComponentScan`

Searches for Spring components.

The current Spring Boot interview material specifically highlights this annotation and its internal components as core questions. ([InterviewBit](#))

Q14. How does Spring Boot Auto-Configuration work?

This is a **product-company-level follow-up**.

Suppose you add:

```
xml
spring-boot-starter-web
```

Spring Boot sees relevant classes on the classpath and configures appropriate infrastructure.

Conceptually:

```
text
Dependencies
    ↓
Classpath detection
    ↓
Auto-configuration conditions
    ↓
Matching configuration
    ↓
Beans created
```

Auto-configuration is generally conditional.

Examples of conditions include:

- class exists
- bean does not already exist
- property exists
- application type matches

Important interview statement

Auto-configuration does not mean Spring Boot blindly creates everything. It applies conditional configuration based on the application's classpath, existing beans and configuration.

Q15. What are Spring Boot Starter dependencies?

Starters are convenient dependency bundles.

Example:

```
xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Instead of manually choosing many compatible dependencies, a starter provides a curated set.

Examples:

```
text
starter-web
starter-data-jpa
starter-security
starter-test
starter-validation
```

Q16. Embedded server?

Spring Boot applications can run with an embedded servlet container.

For example, depending on the stack/configuration, the application can start its web server as part of application startup.

Therefore:

```
bash
java -jar application.jar
```

can launch the application without requiring you to deploy a WAR into a separately managed application server.

Q17. Where do you configure server port?

Typically:

```
properties
server.port=8081
```

or:

```
yaml
server:
  port: 8081
```

This exact type of question has appeared in reported Target interviews. ([GeeksforGeeks](#))

Q18. How do you configure different environments?

Use profiles.

Example:

```
text
application.yml
application-dev.yml
application-qa.yml
application-prod.yml
```

Activate:

```
properties
spring.profiles.active=dev
```

Spring Profiles allow configuration/components to be activated for specific environments. ([Home](#))

Interview scenario

How would you prevent production DB credentials from being stored in Git?

Don't hard-code them.

Use:

- environment variables
- secret manager
- deployment platform secrets
- external configuration

Spring Boot supports externalized configuration through properties/YAML, environment variables and command-line arguments. ([Home](#))

Q19. `@Value` vs `@ConfigurationProperties`

`@Value`

Good for individual values:

```
java
@Value("${payment.timeout}")
private int timeout;
```

`@ConfigurationProperties`

Better for grouped configuration:

```
yaml
payment:
  timeout: 5000
  retry-count: 3
  enabled: true
```

Then bind it to a structured configuration object.

Product-company answer

I prefer @ConfigurationProperties for related configuration because it provides structured, type-safe configuration binding and scales better than having many scattered @Value fields.

PART 3 — REST API

Q20. Explain request flow in Spring Boot

Very important.

Suppose:

```
http
GET /users/10
```

Flow:

```
text
Client
  ↓
HTTP Request
  ↓
Servlet Filter
  ↓
DispatcherServlet
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
  ↓
Repository
  ↓
Service
  ↓
Controller
  ↓
JSON Response
```

For a Spring MVC application, `DispatcherServlet` acts as the central front controller.

Q21. `@Controller` vs `@RestController`

`@Controller`

Generally used for MVC controllers/views.

`@RestController`

Equivalent conceptually to:

```
java
@Controller
@ResponseBody
```

It is commonly used for REST APIs because return values are written to the response body.

Q22. `@RequestParam` vs `@PathVariable` vs `@RequestBody`

Path variable

```
http
GET /users/100

java
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id)
```

Request parameter

```
http
GET /users?page=2

java
@RequestParam int page
```

Request body

```
http
POST /users

json
{
  "name": "John"
}

java
@RequestBody UserRequest request
```

Q23. PUT vs POST vs PATCH

POST

Usually create a resource or trigger an operation.

PUT

Usually replaces the representation of a resource and is generally idempotent.

PATCH

Partial update.

Example:

```
text
POST /users
PUT /users/10
PATCH /users/10
DELETE /users/10
```

Q24. What does idempotent mean?

An operation is idempotent if making the same request multiple times has the same intended final effect as making it once.

For example:

```
http
PUT /users/10
```

with the same representation repeatedly should result in the same final resource state.

This becomes important when designing retries in distributed systems.

Q25. How do you handle exceptions globally?

Use:

```
java
@RestControllerAdvice
```

Example:

```
java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
```

```

    public ResponseEntity<?> handle(UserNotFoundException ex) {
        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(ex.getMessage());
    }
}

```

Why?

Instead of:

text

```

Controller 1 → try/catch
Controller 2 → try/catch
Controller 3 → try/catch

```

we centralize error handling.

Q26. What HTTP status codes should you know?

At minimum:

text

```

200 OK
201 Created
204 No Content
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
422 Unprocessable Content
500 Internal Server Error
503 Service Unavailable

```

Know **when** to use them rather than memorizing numbers.

PART 4 — JPA / HIBERNATE

This is one of the biggest areas for a 2-year Spring Boot interview.

Spring Data JPA's purpose is to reduce persistence boilerplate and provide repository abstractions over JPA. ([Home](#))

Q27. What is JPA?

JPA is a Java specification/API for object-relational persistence.

It maps:

```
text
Java Object
    ↓
Database Table
```

Hibernate is a popular implementation of JPA.

Very important distinction

JPA is a specification; Hibernate is an implementation.

Q28. What is Hibernate?

Hibernate is an ORM framework.

ORM = Object Relational Mapping.

Example:

```
java
@Entity
public class User {

    @Id
    private Long id;

    private String name;
}
```

Hibernate maps this entity to a database table.

Q29. `JpaRepository` vs `CrudRepository`

`CrudRepository` provides basic CRUD operations.

`JpaRepository` provides additional JPA-specific functionality and inherits repository capabilities.

Typical:

```
java
public interface UserRepository
    extends JpaRepository<User, Long> {
}
```

Q30. What is Entity?

An entity is a persistent Java object mapped to a database table.

Example:

```
java
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue
    private Long id;

    private String name;
}
```

Q31. What is `@Id`?

It identifies the primary key of the entity.

```
java
@Id
private Long id;
```

Q32. What is lazy vs eager loading?

Suppose:

```
text
Order
↓
Customer
```

Eager

Associated data is loaded immediately.

Lazy

Associated data is loaded when accessed.

Generally, blindly using eager relationships can cause performance issues.

Interview scenario

You fetch 1,000 orders and each order loads its customer separately. What problem could occur?

N+1 query problem.

Q33. What is N+1?

Example:

```
sql
SELECT * FROM orders;
```

Then Hibernate performs:

```
sql
SELECT * FROM customer WHERE id = ?
SELECT * FROM customer WHERE id = ?
SELECT * FROM customer WHERE id = ?
...
```

One query + N additional queries.

Potential solutions include:

- fetch joins
- entity graphs
- optimized queries
- projections
- batching where appropriate

Q34. What is `@Transactional`?

It defines transaction boundaries.

Example:

```
java
@Transactional
public void transferMoney(...) {
    debit();
    credit();
}
```

Conceptually:

```
text
BEGIN
    debit
    credit
COMMIT
```

If appropriate runtime exceptions cause rollback:

```
text
BEGIN
    debit
    credit
    ↓
    exception
    ↓
ROLLBACK
```

Spring Data JPA documentation explains transaction behavior and how repository methods participate in transactions. ([Home](#))

Q35. Where should `@Transactional` generally be placed?

Usually at the **service layer**, because the service represents a business operation that may involve multiple repository operations.

Example:

```
java
@Service
public class TransferService {

    @Transactional
```

```

    public void transfer(...) {
        accountRepository.debit(...);
        accountRepository.credit(...);
    }
}

```

This is a common interview scenario.

Q36. Why doesn't `@Transactional` work sometimes?

A very important advanced question.

One common issue is **self-invocation**.

Example:

```

java
public void methodA() {
    methodB();
}

@Transactional
public void methodB() {
}

```

Calling `methodB()` from inside the same object can bypass the Spring proxy mechanism that normally applies transactional behavior.

Other causes can include:

- incorrect transaction manager
- method visibility/proxy limitations
- exception/rollback expectations
- transaction configuration

This is exactly the sort of question that separates memorization from understanding.

Q37. What is dirty checking?

Hibernate tracks managed entities.

Suppose:

```
java
User user = repository.findById(1L).get();

user.setName("Vaibhav");
```

You don't necessarily need:

```
java
repository.save(user);
```

for the persistence provider to detect the change when the entity is managed inside a transaction.

Hibernate can detect changes and synchronize them with the database during flush.

Q38. What is persistence context?

Think of it as a context in which JPA manages entity instances.

It maintains entity state and provides features such as:

- first-level caching
- identity guarantee within the context
- dirty checking
- entity lifecycle management

PART 5 — SQL

Do **not** skip SQL for product companies.

Reported 2-year interviews have included SQL alongside Java/Spring and coding. ([GeeksforGeeks](#))

You should know:

```
text
SELECT
WHERE
GROUP BY
HAVING
ORDER BY
JOIN
LEFT JOIN
INNER JOIN
SUBQUERY
```



```
CTE  
WINDOW FUNCTIONS  
INDEXES  
TRANSACTIONS  
NORMALIZATION
```

Q39. INNER JOIN vs LEFT JOIN

INNER JOIN

Returns matching records.

```
sql  
  
SELECT *  
  
FROM users u  
  
JOIN orders o  
  ON u.id = o.user_id;
```

Only users with matching orders.

LEFT JOIN

Returns all users, even if no order exists.

```
sql  
  
SELECT *  
  
FROM users u  
  
LEFT JOIN orders o  
  ON u.id = o.user_id;
```

Q40. What is an index?

An index is a database structure used to make data lookup faster.

Example:

```
sql  
  
CREATE INDEX idx_user_email  
  ON users(email);
```

Without index:

text

Potential full scan

With useful index:

text

Index lookup

↓

Relevant rows

But why not index everything?

Indexes consume:

- storage
- memory/cache
- write/update overhead

So indexes are a tradeoff.

Q41. How would you debug a slow API?

This is an excellent product-company scenario.

Don't immediately say:

Add caching.

Instead investigate.

Step 1

Measure API latency.

text

Request

↓

Controller

↓

Service

↓

Repository

↓

DB

Find where time is spent.

Step 2

Check:

- database query time
- slow SQL
- indexes
- N+1 queries
- external API latency
- thread pool
- connection pool
- GC
- CPU
- memory
- network

Step 3

Optimize based on evidence.

Possible solutions:

text

DB indexing
Query optimization
Caching
Pagination
Connection pool tuning
Async processing
Batch operations
API timeouts

That answer is much stronger than:

"I will increase server memory."

PART 6 — CORE JAVA

For 2 years experience, **Java questions are unavoidable.**

Reported interviews at Accenture, Mastercard, Morgan Stanley and others show extensive Core Java questioning. ([GeeksforGeeks](https://www.geeksforgeeks.org/))

Q42. Explain OOP principles

Four major principles:

Encapsulation

Hide internal state and expose controlled behavior.

Inheritance

Reuse/extend behavior.

Polymorphism

Same interface, different implementations.

```
java
Payment p = new CreditCardPayment();
p.pay();
```

Abstraction

Expose what an object does without exposing unnecessary implementation details.

Q43. Interface vs abstract class

Interface	Abstract Class
Defines contract	Can define contract + shared implementation
Multiple interfaces can be implemented	One class can extend one class
No normal instance state requirement	Can maintain instance state
Supports default/static methods	Supports constructors

Don't just memorize the table.

Know **when you'd use each**.

Q44. Why is String immutable?

Important reasons include:

- security

- string pool
- thread safety
- cached hash code
- predictable behavior

Example:

```
java
String s = "hello";
s.concat(" world");
```

The original string is unchanged.

A new String is produced.

Q45. String vs StringBuilder vs StringBuffer

String

Immutable.

StringBuilder

Mutable and generally preferred for single-threaded string construction.

StringBuffer

Mutable and synchronized; historically useful for thread-safe string mutation but often less preferred than `StringBuilder` when synchronization isn't needed.

Q46. HashMap internal working

Very common in product interviews.

Conceptually:

```
text
key
↓
hash
↓
bucket
↓
entry
```

When you do:

```
java
map.put(key, value);
```

Java calculates a hash and determines a bucket.

If multiple keys map to the same bucket, collision handling is required.

Modern Java HashMap can use tree-based structures in heavily collided buckets under certain conditions.

Follow-ups

You must know:

- `hashCode()`
- `equals()`
- collision
- resizing
- capacity
- load factor
- why mutable keys are dangerous

Q47. HashMap vs ConcurrentHashMap

HashMap

Not designed for concurrent modification by multiple threads.

ConcurrentHashMap

Designed for concurrent access with thread-safety mechanisms and better concurrency characteristics than simply synchronizing an entire map.

Morgan Stanley's reported 2-year interview specifically included HashMap vs ConcurrentHashMap. ([GeeksforGeeks](#))

Q48. Why override equals and hashCode together?

Contract:

If:

```
java
a.equals(b) == true
```

then:

```
java
a.hashCode() == b.hashCode()
```

must be true.

Otherwise hash-based collections can behave incorrectly.

Example:

```
java
HashMap<User, String>
```

If you override `equals()` incorrectly without corresponding `hashCode()`, lookup can fail unexpectedly.

PART 7 — MULTITHREADING

This area is frequently used to distinguish 2-year candidates.

Q49. What is a thread?

A thread is an execution path within a process.

Java applications can execute multiple tasks concurrently using multiple threads.

Q50. Process vs Thread

Process

Independent execution environment with its own memory space.

Thread

Execution unit within a process that shares process resources.

Q51. `synchronized`?

Used to control concurrent access to shared state.

Example:

```
java
public synchronized void increment() {
```

```
count++;  
}
```

It provides mutual exclusion around synchronized regions and participates in Java's memory visibility guarantees.

Q52. What is `volatile`?

`volatile` primarily guarantees visibility of writes between threads and imposes ordering constraints.

Example:

```
java  
private volatile boolean running = true;
```

If one thread changes it, another thread can observe the updated value.

Critical follow-up

Does `volatile` make:

```
java  
count++;
```

atomic?

No.

Because:

```
text  
read  
+  
increment  
+  
write
```

is multiple operations.

Q53. What is race condition?

When multiple threads access shared state and the result depends on timing/order of execution.

Example:


```
java
count++;
```

executed concurrently without appropriate synchronization.

Q54. `synchronized` vs Lock

Know:

```
text
synchronized
ReentrantLock
ReadWriteLock
StampedLock
```

At 2 years, understand why a lock abstraction can be useful:

- explicit lock/unlock
- tryLock
- interruptible lock acquisition
- more flexible coordination

Q55. ExecutorService?

Instead of manually creating threads:

```
java
new Thread(...).start();
```

use a thread pool:

```
java
ExecutorService executor =
    Executors.newFixedThreadPool(10);
```

Advantages:

- thread reuse
- controlled concurrency
- task management
- less thread creation overhead

PART 8 — SPRING SECURITY

Security is increasingly important for backend roles.

Spring Security provides authentication, authorization and protection against common attacks.
([Home](#))

Q56. Authentication vs Authorization

Authentication

Who are you?

Authorization

What are you allowed to do?

Example:

```
text
Login
↓
Authentication
↓
User identified
↓
Authorization
↓
Can user access /admin?
```

Q57. What is JWT?

JWT = JSON Web Token.

A JWT commonly contains:

```
text
Header
Payload
Signature
```

The client sends:

```
http
```

```
Authorization: Bearer <token>
```

The server validates the token and establishes the authenticated identity/authorities.

Spring Security's current resource-server implementation uses components such as `BearerTokenAuthenticationFilter`, `AuthenticationManager`, `JwtAuthenticationProvider` and `JwtDecoder` for JWT authentication. ([Home](#))

Q58. JWT authentication flow

```
text
```

```
Client
```

```
↓
```

```
Login
```

```
↓
```

```
Authentication Server
```

```
↓
```

```
JWT
```

```
↓
```

```
Client stores token
```

```
↓
```

```
API request
```

```
Authorization: Bearer JWT
```

```
↓
```

```
Security Filter
```

```
↓
```

```
JWT validation
```

```
↓
```

```
Authentication
```

```
↓
```

```
Authorization
```

```
↓
```

```
Controller
```

Q59. 401 vs 403

401

Authentication missing/invalid.

"I don't know who you are."

403

Authenticated but not permitted.

"I know who you are, but you can't do this."

PART 9 — MICROSERVICES

This is **very important** for product companies.

Q60. Monolith vs Microservices

Monolith

text

Application



Microservices

text

User Service

↓

Order Service

↓

Payment Service

↓

Notification Service

Each service can be independently deployed/scaled.

But don't say:

Microservices are always better.

That's a weak answer.

They introduce:

- network failures
- distributed transactions
- observability complexity
- deployment complexity
- eventual consistency
- service discovery
- data consistency challenges

Q61. What is API Gateway?

Acts as a gateway between clients and backend services.

Potential responsibilities:

```
text
Client
  ↓
API Gateway
  ├── Authentication
  ├── Routing
  ├── Rate limiting
  ├── Logging
  └── Request policies
  ↓
Microservices
```

Q62. What is service discovery?

In dynamic environments, service instances may change.

Instead of hardcoding:

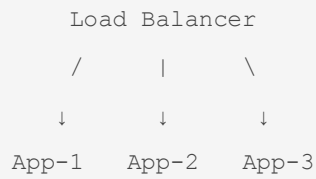
```
text
http://10.20.10.3:8080
```

services can discover available instances through a discovery mechanism.

Q63. What is load balancing?

Distributes traffic across multiple instances.

text



Benefits:

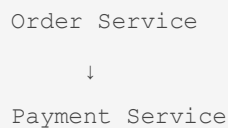
- scalability
- availability
- traffic distribution

Target's 2-year interview report explicitly included load balancing. ([GeeksforGeeks](#))

Q64. What is Circuit Breaker?

Suppose:

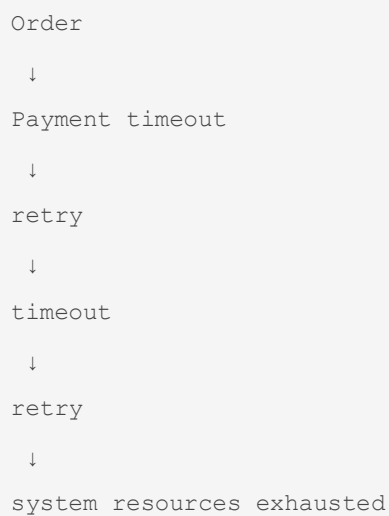
text



Payment is down.

Without protection:

text



Circuit breaker prevents continuous calls.

States:

```
text
CLOSED
  ↓ failures
OPEN
  ↓ wait
HALF_OPEN
  ↓ test
CLOSED / OPEN
```

This was explicitly reported in Target and Tech Mahindra interviews. ([GeeksforGeeks](#))

Q65. What is distributed transaction?

Suppose:

```
text
Order Service
  ↓
Payment Service
  ↓
Inventory Service
```

All need to succeed.

In a monolith you might have one database transaction.

In microservices, each service can have its own database.

Therefore global transaction management becomes difficult.

Possible approaches include:

- Saga pattern
- eventual consistency
- compensating actions
- carefully designed workflows

PART 10 — KAFKA

If Kafka is on your resume, expect questions.

Q66. What is Kafka?

Kafka is a distributed event streaming platform.

Basic architecture:

```
text
Producer
  ↓
Kafka Topic
  ↓
Partition
  ↓
Consumer
```

Multiple consumers can process events.

Q67. Why Kafka instead of REST?

REST:

```
text
Service A → synchronous → Service B
```

Kafka:

```
text
Service A
  ↓
Event
  ↓
Kafka
  ↓
Service B
```

Kafka can help with:

- asynchronous processing
- decoupling
- event-driven architecture
- buffering
- replaying events

But don't say Kafka is automatically better than REST.

Q68. What is a Kafka partition?

A topic can have multiple partitions.

Example:

```
text
Orders Topic

Partition 0
Partition 1
Partition 2
```

Partitions enable parallelism and distribution.

PART 11 — REDIS / CACHING

Q69. Why use caching?

Suppose DB query takes:

```
text
200 ms
```

But cached lookup takes:

```
text
5 ms
```

Caching can reduce:

- latency
- database load
- repeated computation

Architecture:

```
text
Client
↓
Service
↓
Redis
```

```
↓ cache miss
Database
```

Q70. Cache-aside pattern

Most common conceptual pattern:

```
text
Read
↓
Check cache
↓
Hit → return
↓
Miss
↓
DB
↓
put into cache
↓
return
```

Follow-up

What if DB updates but cache contains old data?

Cache invalidation / stale data problem.

This is one of the classic distributed-systems interview topics.

PART 12 — ACTUATOR / PRODUCTION

Q71. What is Spring Boot Actuator?

Actuator exposes production-oriented monitoring and management endpoints.

Typical concerns:

```
text
Health
Metrics
Info
```

```
Environment
Beans
Mappings
```

Use carefully in production and secure sensitive endpoints.

Q72. How would you monitor a production Spring Boot application?

Think in layers:

```
text
Application
├─ Logs
├─ Metrics
├─ Traces
└─ Health checks

Infrastructure
├─ CPU
├─ Memory
├─ Disk
└─ Network

Database
├─ Connection pool
├─ Query latency
└─ Locks
```

A mature answer mentions:

- metrics
- logs
- distributed tracing
- alerts
- dashboards
- correlation/request IDs

PART 13 — TESTING

Q73. Unit test vs Integration test

Unit test

Tests a small component in isolation.

Example:

```
text
Service
↓
mock Repository
```

Integration test

Tests interaction between components.

Example:

```
text
Controller
↓
Service
↓
Repository
↓
Database/Test DB
```

Q74. Mockito `@Mock` vs `@InjectMocks`

```
java
@Mock
private UserRepository repository;

@InjectMocks
private UserService service;
```

`@Mock` creates a mock.

`@InjectMocks` attempts to construct/inject the mocks into the target object.

Q75. What should you test in a REST API?

At minimum:

text

```
200 success
400 validation failure
401 unauthenticated
403 unauthorized
404 not found
500 unexpected error
```

Also:

- request validation
- business rules
- edge cases
- database interactions
- security
- serialization

PART 14 — DSA

This is where many Java developers underestimate product companies.

Target reported DSA in multiple rounds, including arrays, graphs/DP and additional coding questions. ([GeeksforGeeks](https://www.geeksforgeeks.org/))

For 2 years, prepare:

Arrays

- two sum
- maximum subarray
- sliding window
- prefix sum
- merge intervals
- top K elements

Strings

- anagram
- palindrome
- longest substring
- frequency counting

HashMap

- frequency problems
- duplicate detection
- two sum
- grouping

Linked List

- reverse list
- cycle detection
- nth node from end
- merge sorted lists

Stack/Queue

- valid parentheses
- monotonic stack
- sliding window maximum

Trees

- DFS
- BFS
- height
- diameter
- BST

Graph

- BFS
- DFS
- connected components
- shortest path basics
- number of islands

Dynamic Programming

At minimum:

- climbing stairs
- house robber
- coin change
- basic 1D DP

PART 15 — MOST IMPORTANT SCENARIO QUESTIONS

These are the questions I would **especially drill** for top product companies.

Scenario 1

Your API suddenly becomes slow in production. What will you do?

Expected thought process:

```
text
Measure
↓
Find bottleneck
↓
Application?
↓
Database?
↓
External API?
↓
Network?
↓
Thread pool?
↓
Connection pool?
↓
CPU/memory?
↓
Fix bottleneck
↓
Measure again
```

Scenario 2

API works for 100 users but fails under 10,000 concurrent users.

Discuss:

- horizontal scaling
- load balancing
- connection pool
- thread pool
- database bottleneck

- caching
- rate limiting
- async processing
- stateless services

Scenario 3

Database CPU is 100%. What will you check?

Strong answer:

```
text
Slow queries
Indexes
Execution plans
N+1
Missing pagination
Large joins
Connection count
Long transactions
Locks
Unnecessary queries
```

Scenario 4

Payment API is down. Your Order API depends on it.

Discuss:

```
text
Timeout
Retry
Exponential backoff
Circuit breaker
Fallback
Idempotency
Dead-letter/event mechanism
Observability
```

Scenario 5

User clicks "Pay" twice.

What can happen?

text

Payment 1 → successful

Payment 2 → successful

User gets charged twice.

Solution:

Idempotency key.

Example:

http

POST /payments

Idempotency-Key: abc123

Server remembers the result for that logical operation.

This is an **excellent product-company scenario**.

PART 16 — PROJECT DEEP DIVE

This is probably the most important part for you personally.

At 2 years experience, interviewers often take your resume/project and drill down.

Infosys reported project, Java, Spring Boot, multithreading, SQL and JUnit questions in its 2-year Java interview; Aptio similarly went deep into projects plus Spring/JPA/Java internals.

([GeeksforGeeks](#))

You should be able to answer:

1.

Explain your project.

Use:

text

Problem

↓

Architecture

↓

Your responsibility

↓

Technologies

↓
API flow
↓
Database
↓
Challenges
↓
Performance
↓
Production issues
↓
Improvements

Project question #2

Why did you use Spring Boot?

Don't say:

Because it is easy.

Say:

We used Spring Boot because it reduced configuration through auto-configuration and starters, provided embedded application infrastructure, integrated naturally with Spring Data and Spring Security, and allowed us to build and deploy the service as a stand-alone application.

Project question #3

Why did you use microservices?

Don't say:

Because microservices are modern.

Say:

We separated services around business capabilities so that they could be developed and deployed independently and scaled independently where necessary. However, this introduced distributed-system concerns such as network failures, consistency and observability, so the decision was based on those trade-offs rather than simply choosing microservices by default.

Project question #4

What was the hardest problem you solved?

Prepare **2 real stories**.

Structure:

```
text
Problem
↓
Impact
↓
Investigation
↓
Root cause
↓
Solution
↓
Result
```

PART 17 — QUESTIONS YOU SHOULD EXPECT FROM YOUR RESUME

If you have:

Spring Boot

Expect:

```
text
@SpringBootApplication
Auto configuration
Bean lifecycle
Dependency injection
Profiles
Configuration
Actuator
REST
Exception handling
```

JPA

Expect:

text

Entity
Relationships
Lazy/Eager
N+1
Transactions
Persistence context
Dirty checking
Cascade
Fetch join

Microservices

Expect:

text

API Gateway
Service discovery
Load balancing
Circuit breaker
Retries
Timeout
Kafka
Distributed transaction
Saga

Docker

Expect:

text

Image vs container
Dockerfile
Port mapping
Volumes
Networks
Multi-stage builds

AWS

Expect questions about whatever you actually wrote.

Never put technology on your resume that you can't explain for 10–15 minutes.

□ TOP 50 QUESTIONS TO MEMORIZE/UNDERSTAND

If your interview is very soon, prioritize these:

Spring

1. What is Spring?
2. What is IoC?
3. What is DI?
4. Constructor vs field injection?
5. What is Bean?
6. Bean lifecycle?
7. Bean scopes?
8. `@Component` vs `@Service` vs `@Repository`
9. `@Autowired`
10. `@Qualifier`
11. `@Primary`
12. Spring vs Spring Boot
13. What is `@SpringBootApplication`?
14. Auto-configuration?
15. Starter dependencies?
16. Profiles?
17. `@Value` vs `@ConfigurationProperties`

REST

1. `@Controller` vs `@RestController`
2. `@RequestParam` vs `@PathVariable`
3. `@RequestBody`
4. GET vs POST
5. PUT vs PATCH
6. Idempotency
7. HTTP status codes
8. Global exception handling
9. Request flow through Spring MVC

JPA

1. JPA vs Hibernate
2. `JpaRepository`
3. Entity lifecycle

4. Lazy vs eager
5. N+1
6. Persistence context
7. Dirty checking
8. `@Transactional`
9. Transaction propagation
10. Transaction rollback
11. Self-invocation problem

Java

1. OOP
2. Interface vs abstract class
3. String immutability
4. HashMap internals
5. HashMap vs ConcurrentHashMap
6. equals/hashCode
7. ArrayList vs LinkedList
8. HashSet internals
9. `synchronized`
10. `volatile`
11. `ExecutorService`

Microservices

1. Monolith vs microservices
2. Circuit breaker / retry / timeout / load balancing

Then move into Kafka, Redis, Security, Docker, SQL and LLD.

□ The "WHY?" technique

This is the biggest change I'd recommend in your preparation.

Don't prepare:

| *What is @Transactional?*

Prepare:

| *Why do we need transactions?*

Then:

How does Spring implement it?

Then:

What happens when the method throws an exception?

Then:

Which exceptions cause rollback?

Then:

What happens if one transactional method calls another?

Then:

What happens during self-invocation?

Then:

What if the database operation takes 30 seconds?

That is how a product-company interviewer thinks.

□ Product-company interview ladder

You should be able to handle this progression:

text

LEVEL 1

What is Spring Boot?

↓

LEVEL 2

How does `@SpringBootApplication` work?

↓

LEVEL 3

Explain auto-configuration.

↓

LEVEL 4

How does Spring decide which beans to create?

↓

LEVEL 5

Two beans implement the same interface. What happens?

↓

LEVEL 6

How would you solve it?

↓

LEVEL 7

How does Spring implement @Transactional?

↓

LEVEL 8

Why can self-invocation break transactional behavior?

↓

LEVEL 9

Your transaction calls an external payment service.
Should you keep the DB transaction open?

↓

LEVEL 10

How would you redesign this for a production distributed system?

Level 1–3 = basic candidate.

Level 4–6 = good 2-year candidate.

Level 7–10 = product-company-ready thinking.

□ Sources I used

I checked current published interview material and actual candidate interview reports rather than relying only on generic knowledge.

- — broad Spring Boot questions, updated July 2025. ([GeeksforGeeks](#))
- — updated February 2026, including auto-configuration, starters and `@SpringBootApplication`. ([InterviewBit](#))
- — actual reported interview questions. ([GeeksforGeeks](#))
- — Java, Spring Boot, load balancing, circuit breaker, Docker, SQL and DSA. ([GeeksforGeeks](#))
- — multi-round technical process. ([GeeksforGeeks](#))
- — deep Spring/JPA/Java/threads/design-pattern questions. ([GeeksforGeeks](#))
- — Core Java, collections, concurrency and coding. ([GeeksforGeeks](#))
- — current JPA/repository reference. ([Home](#))
- — current authentication/authorization/JWT material. ([Home](#))

There is also useful scenario-oriented video material; for example, a Spring Boot scenario/tricky-question interview video explicitly targets experienced candidates and discusses questions associated with large tech-company interviews. ([YouTube](#))

✂ But this is only Part 1

The above is the **high-value master map**, not literally every possible interview question. Trying to dump 500+ questions into one response would make it much harder to study.

For your goal, I recommend turning this into a structured “**2-Year Java + Spring Boot Product Company Interview Bible**” with:

Module 1: 150 Core Java questions + answers **Module 2:** 120 Spring/Spring Boot questions + internal working **Module 3:** 80 JPA/Hibernate questions **Module 4:** 75 REST/API questions **Module 5:** 75 Microservices questions **Module 6:** 50 Spring Security/JWT questions **Module 7:** 50 SQL questions + queries **Module 8:** 50 Multithreading questions **Module 9:** 50 Kafka + Redis questions **Module 10:** 50 Docker/AWS questions **Module 11:** 100 DSA problems specifically for 2-year Java SDE interviews **Module 12:** 50 LLD/system-design scenarios **Module 13:** 100 project/resume follow-up questions **Module 14:** Company-specific patterns for **Amazon, Google, Microsoft, Adobe, Uber, Walmart, Mastercard, Morgan Stanley, Cisco, Salesforce, Atlassian, Oracle, etc.**

Most importantly, for every question I'd structure it as:

Question → Short interview answer → Detailed explanation → Real project example → Follow-up question → Tricky interviewer counter-question → Best answer → Code/diagram where relevant

That format will train you to **answer follow-up questions instead of just memorizing definitions.**

If you're serious about cracking these interviews, the next useful step is to start with **Module 1: 150 Core Java questions for 2-year product-company interviews**, because almost every Spring Boot round eventually comes back to Java internals.

you asked: yes, one b

message time: 2026-08-14 21:07:40

yes, one by one module give me questions and answers.

The above is the **high-value master map**, not literally every possible interview question. Trying to dump 500+ questions into one response would make it much harder to study.

For your goal, I recommend turning this into a structured “**2-Year Java + Spring Boot Product Company Interview Bible**” with:

Module 1: 150 Core Java questions + answers **Module 2:** 120 Spring/Spring Boot questions + internal working **Module 3:** 80 JPA/Hibernate questions **Module 4:** 75 REST/API questions **Module 5:** 75 Microservices questions **Module 6:** 50 Spring Security/JWT questions **Module 7:** 50 SQL questions + queries **Module 8:** 50 Multithreading questions **Module 9:** 50 Kafka + Redis questions **Module 10:** 50 Docker/AWS questions **Module 11:** 100 DSA problems specifically for 2-year Java SDE interviews **Module 12:** 50 LLD/system-design scenarios **Module 13:** 100 project/resume follow-up questions **Module 14:** Company-specific patterns for **Amazon, Google, Microsoft, Adobe, Uber, Walmart, Mastercard, Morgan Stanley, Cisco, Salesforce, Atlassian, Oracle, etc.**

Most importantly, for every question I'd structure it as:

Question → Short interview answer → Detailed explanation → Real project example → Follow-up question → Tricky interviewer counter-question → Best answer → Code/diagram where relevant

That format will train you to **answer follow-up questions instead of just memorizing definitions.**

chatgpt response: Perfect. W

Perfect. We'll do it **one module at a time**, and I'll keep the format consistent throughout the Interview Bible.

Module 1 — Core Java

150 Interview Questions & Answers for 2-Year Product-Company Candidates

This module is intentionally **not beginner-only**. The progression will be:

Basics → Collections → OOP → String → Java 8+ → Exceptions → JVM → Memory → Multithreading → Advanced Java → Coding-oriented questions → Product-company follow-ups

For each question:

Question → Short interview answer → Detailed explanation → Example → Follow-up → Tricky counter-question → Best answer → Code

I'll also mark questions with:

- ☐ **Very frequently asked**
- **★ Important**
- ☐ **Product-company depth**
- ☐ **Common trap**

SECTION A — JAVA FUNDAMENTALS & OOP

Q1. What are the main features of Java? ☐

Short interview answer

Java is a high-level, object-oriented, class-based programming language designed to be platform-independent through the JVM.

Important features include:

- Object-oriented
- Platform independent
- Robust
- Secure
- Multithreaded
- Automatic memory management
- Portable
- High performance through JIT compilation
- Rich standard library

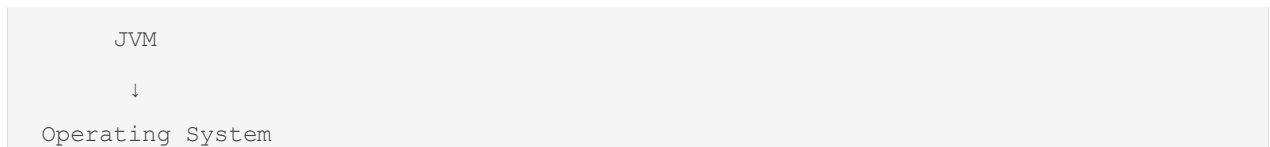
Detailed explanation

The famous Java principle is:

Write Once, Run Anywhere.

Java source code:

```
text
Java source code
    ↓
    javac
    ↓
    Bytecode (.class)
    ↓
```



The `.class` bytecode is not directly executed by the operating system.

The JVM executes it.

That's why the same Java bytecode can run on different operating systems as long as an appropriate JVM exists.

Real project example

If you build:

```
text
Spring Boot application
```

you compile it into bytecode and package it, commonly as a JAR.

The same application can run on:

```
text
Linux
Windows
Docker
Cloud VM
Kubernetes
```

provided the required Java runtime is available.

Follow-up

Is Java completely platform independent?

Best answer

Java bytecode is platform-independent, but the JVM itself is platform-dependent because each operating system requires its own JVM implementation.

Tricky counter-question ☐

Then why do we install Java separately on Windows and Linux?

Because the JVM implementation is OS-specific.

```
text
Windows → Windows JVM
```

```
Linux    → Linux JVM
macOS    → macOS JVM
```

Q2. What is the difference between JDK, JRE and JVM? □ □ □

This is one of the most common Java interview questions.

Short interview answer

- **JVM** executes Java bytecode.
- **JRE** provides the JVM plus runtime libraries.
- **JDK** provides development tools plus the runtime needed to run Java applications.

Conceptually:

```
text
JDK
├─ Development tools
│   ├── javac
│   ├── javadoc
│   └── jar
└─ Runtime
    ├── JVM
    └── Java libraries
```

Detailed explanation

JVM

Responsible for executing bytecode.

JRE

Provides everything needed to run Java applications.

JDK

Provides everything needed to **develop and run** Java applications.

Example

When you execute:

```
bash
javac Hello.java
```

you need a JDK because `javac` is a development tool.

When you run:

```
bash
java Hello
```

the JVM executes the compiled bytecode.

Tricky question

Is JRE still separately distributed in modern Java versions?

Don't blindly repeat the old statement:

"JDK = JRE + JVM."

That is useful for understanding the historical architecture, but modern Java distributions have changed how these components are packaged and distributed.

For interviews, say:

Conceptually, JRE represents the runtime environment and JDK represents the development environment, but modern Java distributions don't necessarily provide a separately packaged JRE like older Java releases did.

That's a much stronger answer.

Q3. Why is Java platform independent? ☐

Short answer

Because Java source code is compiled into **platform-independent bytecode**, which is executed by a platform-specific JVM.

```
text
Source
  ↓
javac
  ↓
Bytecode
  ↓
JVM
  ↓
OS
```

Example

You compile:

```
java
public class Test {
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

The generated bytecode can run on different operating systems.

Follow-up

Is the JVM platform independent?

No.

The JVM implementation is platform dependent.

The **bytecode** is platform independent.

Q4. Why is Java not considered a purely object-oriented language? ★

Short answer

Because Java supports primitive data types such as:

```
java
int
char
boolean
byte
short
long
float
double
```

These are not objects.

For example:

```
java
int age = 25;
```

`age` is a primitive value rather than an object.

Java provides wrapper classes:

```
java
Integer
Character
Boolean
```

when an object representation is required.

Follow-up

Does Java automatically convert primitive types to wrapper objects?

Yes, through **autoboxing**.

```
java
Integer x = 10;
```

Conceptually:

```
java
Integer x = Integer.valueOf(10);
```

Q5. Explain the four pillars of OOP. ☐ ☐ ☐

The four major principles are:

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

1. Encapsulation

Bundling state and behavior together while controlling access to internal state.

```
java
public class Account {

    private double balance;

    public double getBalance() {
        return balance;
    }
}
```



```

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }
}

```

The caller cannot directly modify:

```

java
    balance

```

because it is private.

2. Abstraction

Expose necessary behavior while hiding implementation details.

```

java
    interface PaymentService {

        void pay(double amount);

    }

```

The caller knows:

```

text
    pay()

```

but doesn't need to know how the payment is processed internally.

3. Inheritance

A class can derive behavior from another class.

```

java
    class Animal {
        void eat() {
            System.out.println("Eating");
        }
    }

```

```
class Dog extends Animal {  
}
```

Dog inherits eat().

4. Polymorphism

One interface/reference can represent different implementations.

```
java  
  
PaymentService payment;  
  
payment = new StripePayment();  
payment.pay();  
  
payment = new RazorpayPayment();  
payment.pay();
```

Same interface:

```
text  
  
pay()
```

different behavior.

Q6. Encapsulation vs abstraction? ☐

This is a common follow-up.

Encapsulation	Abstraction
Controls access to data/behavior	Hides implementation complexity
Usually achieved using access modifiers	Usually achieved using interfaces/abstract classes
Focuses on protecting state	Focuses on exposing essential behavior

Simple way to remember

Encapsulation:

"How do I protect this?"

Abstraction:

"What should I expose?"

Q7. What is inheritance? ☆

Inheritance allows one class to acquire properties and behavior from another class.

```
java
class Vehicle {

    void start() {
        System.out.println("Vehicle started");
    }
}

class Car extends Vehicle {
}
```

Now:

```
java
Car car = new Car();
car.start();
```

works because `Car` inherits `start()`.

Why is inheritance useful?

It can provide:

- code reuse
- specialization
- polymorphism

But what's the problem?

Inheritance creates tight coupling.

Modern design often prefers:

Composition over inheritance

when inheritance does not represent a genuine "is-a" relationship.

Q8. What is polymorphism? □ □

Polymorphism means:

One interface/reference, multiple implementations or forms of behavior.

There are two major forms discussed in Java interviews.

Compile-time polymorphism

Method overloading.

```
java
void calculate(int a) {}

void calculate(int a, int b) {}
```

Runtime polymorphism

Method overriding.

```
java
class Payment {
    void pay() {
    }
}

class CreditCardPayment extends Payment {

    @Override
    void pay() {
        System.out.println("Credit Card");
    }
}

java
Payment payment = new CreditCardPayment();

payment.pay();
```

The runtime implementation is:

```
text
CreditCardPayment.pay()
```

Q9. Overloading vs overriding ☐ ☐ ☐

Overloading	Overriding
Same method name	Same method signature
Different parameters	Same parameters
Compile-time	Runtime
Usually same class	Parent-child relationship
Return type alone cannot overload	Covariant return types are allowed

Example: Overloading

```
java
class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

}
```

Example: Overriding

```
java
class Animal {

    void sound() {
        System.out.println("Animal sound");
    }

}

class Dog extends Animal {

    @Override
    void sound() {
        System.out.println("Bark");
    }

}
```

Q10. Can we overload a method by changing only its return type? ☐

No.

This is invalid:

```
java
int calculate() {
    return 10;
}

String calculate() {
    return "10";
}
```

Why?

Because the compiler cannot distinguish methods based only on return type.

Method signature overloading depends on the method name and parameter list, not return type alone.

Q11. Can a static method be overridden? ☐

No, not in the runtime-polymorphism sense.

Static methods belong to the class rather than being dynamically dispatched based on the object.

They can be **hidden** in a subclass.

```
java
class Parent {

    static void show() {
        System.out.println("Parent");
    }
}

class Child extends Parent {

    static void show() {
        System.out.println("Child");
    }
}
```

```
java
Parent obj = new Child();

obj.show();
```

Output:

```
text
Parent
```

because static method resolution is based on the reference type.

Interview phrase

Static methods are hidden, not overridden.

Q12. Can private methods be overridden?

No.

Private methods are not accessible to subclasses and are not inherited in the normal overriding sense.

Example:

```
java
class Parent {

    private void test() {

    }

}
```

A subclass can define another method called:

```
java
test()
```

but it is not overriding the parent's private method.

Q13. Can constructors be inherited?

No.

Constructors belong to the class in which they are declared.

A subclass can invoke a parent constructor using:

```
java
super();
```

but it does not inherit the constructor.

Q14. What is the difference between `this` and `super`? ☐

`this`

Refers to the current object.

```
java
this.name = name;
```

`super`

Refers to the parent-class portion of the current object.

```
java
super.calculate();
```

or:

```
java
super();
```

to invoke a parent constructor.

Q15. What is an abstract class?

A class declared with:

```
java
abstract
```

that can contain:

- abstract methods
- concrete methods
- fields
- constructors
- static methods
- instance methods

Example:


```

java

abstract class Payment {

    abstract void pay();

    void log() {
        System.out.println("Payment initiated");
    }
}

```

You cannot directly instantiate:

```

java

new Payment();

```

Q16. What is an interface? ☐

An interface defines a contract that classes can implement.

```

java

interface Payment {

    void pay();

}

```

Implementation:

```

java

class CardPayment implements Payment {

    @Override
    public void pay() {
        System.out.println("Card payment");
    }

}

```

Interfaces can also contain modern Java features such as:

- default methods
- static methods
- private methods

depending on the Java version and feature being used.

Q17. Abstract class vs interface ☐☐☐

This is a **must-know product-company question**.

Abstract class	Interface
<code>extends</code>	<code>implements</code>
A class can extend one class	A class can implement multiple interfaces
Can have instance state	Interfaces primarily define contracts
Can have constructors	No instance constructors
Can contain concrete methods	Can contain default/static/private methods
Useful for shared base behavior	Useful for contracts/capabilities

Practical example

Suppose you have:

```
text
Payment
```

and implementations:

```
text
CreditCard
UPI
Wallet
NetBanking
```

An interface is often appropriate:

```
java
interface PaymentProcessor {
    PaymentResult process(PaymentRequest request);
}
```

Then:

```
java
class UpiPaymentProcessor
```

```
implements PaymentProcessor {  
}
```

This also works naturally with Spring dependency injection.

Q18. Why does Java not support multiple inheritance of classes?

Because it can create ambiguity.

Imagine:

```
text  
  
    A  
  /  \  
B      C  
  \  /  
    D
```

If both **B** and **C** define:

```
java  
void show()
```

which implementation should **D** inherit?

This is commonly called the **diamond problem**.

Java avoids multiple inheritance of classes but allows multiple interfaces.

Q19. What is composition? □

Composition means one class contains another object rather than inheriting from it.

```
java  
class OrderService {  
  
    private final PaymentService paymentService;  
  
    OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
}
```

OrderService **has a** **PaymentService**.

This is:

Has-a relationship.

Inheritance is:

Is-a relationship.

Product-company follow-up

Why is composition often preferred?

Because it generally gives:

- lower coupling
- easier testing
- easier replacement of behavior
- more flexible design

And this connects directly to Spring dependency injection.

Q20. What is the difference between IS-A and HAS-A?

IS-A

Inheritance.

```
java
class Dog extends Animal
```

Dog **is an** Animal.

HAS-A

Composition.

```
java
class OrderService {

    private PaymentService paymentService;

}
```

OrderService **has a** PaymentService.

Q21. What is an immutable class? ☐ ☐

An immutable object cannot change its observable state after construction.

`String` is the classic Java example.

Example:

```
java
public final class Employee {

    private final String name;
    private final int age;

    public Employee(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }
}
```

Characteristics:

- class often declared `final`
- fields `private final`
- initialized through constructor
- no setters
- mutable internal state must be protected

Why immutable objects are useful

They are easier to reason about and can be safer to share across threads.

Q22. How do you create a truly immutable class? □

This is a very good follow-up.

Suppose:

```
java
class Employee {

    private final List<String> skills;

}
```

Simply making the reference `final` does **not** make the list immutable.

This is dangerous:

```
java
public List<String> getSkills() {

    return skills;

}
```

The caller can modify the list.

Instead, return an unmodifiable/copy representation.

For example:

```
java
public List<String> getSkills() {

    return List.copyOf(skills);

}
```

And defensively copy incoming mutable data where necessary.

Important concept

final reference $\neq$ immutable object.

Q23. What is the difference between `final`, `finally`, and `finalize()`?

☐ ☐ ☐

`final`

Used with:

- variables
- methods
- classes

Example:

```
java
final int x = 10;
```

Cannot reassign `x`.

```
java
final class Payment {
}
```

Cannot extend the class.

```
java
final void pay() {
}
```

Cannot override the method.

`finally`

Used with exception handling.

```
java
try {
    // code
} catch (Exception e) {
    // handle
} finally {
    // cleanup
}
```

`finalize()`

This is an old object-finalization mechanism that has been deprecated for removal and should **not** be used for resource management.

For modern Java, prefer explicit resource management such as:

```
java
try-with-resources
```

Interview trap ☐

If an interviewer asks:

"How do you use `finalize` for cleanup?"

Don't recommend it.

Say:

finalize() is obsolete/deprecated and should not be relied upon for cleanup. I would use try-with-resources or explicit lifecycle management instead.

Q24. What is a constructor?

A constructor initializes an object when it is created.

```
java
class User {

    private String name;

    User(String name) {
        this.name = name;
    }
}
```

Usage:

```
java
User user = new User("John");
```

Important characteristics:

- same name as class
- no return type
- invoked during object creation
- can be overloaded
- not inherited
- cannot be overridden

Q25. What is constructor chaining?

One constructor calls another constructor.

Using:

```
java
this(...)
```

Example:


```

java
class User {

    private String name;
    private int age;

    User() {
        this("Unknown", 0);
    }

    User(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

```

Parent constructor invocation uses:

```

java
super(...)

```

Important rule

`this(...)` or `super(...)` must appear as the first statement of a constructor.

Q26. What happens when you create an object using `new`?

Example:

```

java
User user = new User();

```

At a conceptual level:

```

text
new
↓
memory allocated for object
↓
fields receive default values
↓
constructor executes

```

```
↓  
reference returned
```

For interviews, don't oversimplify this into:

"Object always goes directly into heap and that's it."

Modern JVMs perform sophisticated optimizations such as escape analysis and allocation optimizations.

The important conceptual model is:

Objects are generally associated with heap-managed memory, while local variables/references and execution state are managed as part of thread execution.

Q27. Stack vs Heap? □ □ □

Stack

Each thread has its own execution stack.

It contains things such as:

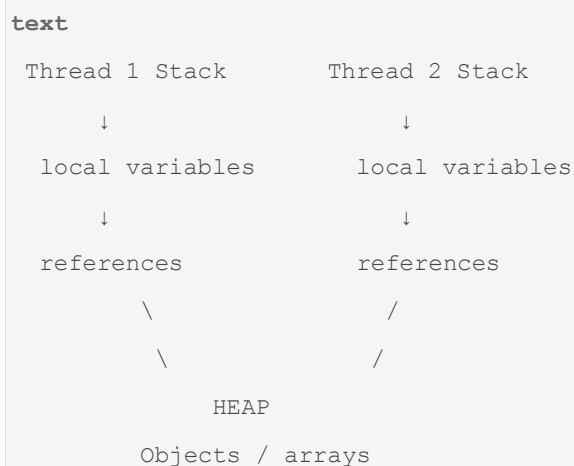
- stack frames
- local variables
- method execution state
- references to objects

Heap

Shared across threads.

Contains dynamically allocated objects managed by the JVM's garbage collector.

Conceptually:



Product-company follow-up

Are primitive values always on the stack?

Don't give an absolute answer.

Their storage depends on context and JVM implementation/optimization.

For example:

```
java
int x = 10;
```

as a local variable is conceptually associated with the current stack frame, but modern JVM optimizations can change physical representation.

Q28. What is garbage collection? ☐ ☐

Garbage collection automatically reclaims memory occupied by objects that are no longer reachable.

Example:

```
java
User user = new User();

user = null;
```

If no other reachable reference exists, the original object may eventually become eligible for garbage collection.

Important

Garbage collection does **not** mean:

"Java immediately deletes the object."

It means:

The object becomes eligible for reclamation when it is no longer reachable, and the JVM's garbage collector determines when/how to reclaim it.

Q29. Can we force garbage collection?

No.

You can request it:

```
java
System.gc();
```

but the JVM is not required to perform GC immediately because of that call.

Interview answer

System.gc() is only a request/hint to the JVM, not a guarantee that garbage collection will happen immediately.

Q30. What is a memory leak in Java?

Java has garbage collection, but memory leaks are still possible.

A memory leak occurs when objects are no longer logically needed but remain reachable, so the garbage collector cannot reclaim them.

Example:

```
java
static List<Object> cache = new ArrayList<>();
```

If objects keep being added and never removed, the list retains references to them.

Therefore:

```
text
No longer needed
    ↓
Still referenced
    ↓
GC cannot reclaim
    ↓
Memory grows
```

Real-world examples

- unbounded caches
- static collections
- listeners not deregistered
- ThreadLocal misuse
- collections retaining old objects

Q31. What is the difference between `==` and `.equals()`? ☐☐☐

``==``

For objects, checks whether two references refer to the same object.

``equals()``

Checks logical equality according to the implementation.

Example:

```
java
String a = new String("Java");
String b = new String("Java");

java
a == b
```

is generally:

```
text
false
```

while:

```
java
a.equals(b)
```

is:

```
text
true
```

because String overrides `equals()` to compare content.

Q32. Why should ``equals()`` and ``hashCode()`` be overridden together?

☐ ☐ ☐

Contract:

If:

```
java
a.equals(b)
```

is `true`, then:

```
java
a.hashCode() == b.hashCode()
```

must also be true.

This matters for:

```
text
HashMap
HashSet
Hashtable
```

Example

Suppose:

```
java
class Employee {

    private int id;

}
```

If two employees with the same ID are logically equal, your `equals()` and `hashCode()` implementations need to reflect that same definition.

Q33. What happens if you override `equals()` but not `hashCode()`? ☐

You can break hash-based collection behavior.

For example:

```
java
Set<Employee> employees = new HashSet<>();
```

If two logically equal employees generate different hash codes, the set may treat them as different entries.

Interview statement

Equal objects must have equal hash codes. The reverse is not necessarily true.

That last sentence is important.

Q34. What is the String Pool? ☐ ☐

Java maintains a pool of interned strings.

Example:

```
java
String a = "Java";
String b = "Java";
```

These literals can refer to the same pooled String object.

Therefore:

```
java
a == b
```

can be:

```
text
true
```

But:

```
java
String c = new String("Java");
```

creates a distinct String object.

So:

```
java
a == c
```

is generally:

```
text
false
```

while:

```
java
a.equals(c)
```

is:

```
text
true
```

Golden interview rule

For String content comparison:

```
java
.equals()
```

not:

```
java
==
```

Q35. Why is String immutable? ☐☐☐

Several reasons.

1. Security

Strings are used for things such as:

```
text
file paths
URLs
class names
credentials
configuration
```

2. String pool

Immutability allows pooled String instances to be safely shared.

3. Thread safety

Immutable objects can be safely shared without synchronization for their immutable state.

4. Hashing

Strings are frequently used as keys in hash-based collections.

If a String changed after being inserted into a HashMap, lookup behavior could become problematic.

Q36. String vs StringBuilder vs StringBuffer ☐☐

Type	Mutable?	Thread synchronization	Typical use
String	No	N/A	Immutable text
StringBuilder	Yes	No	Most string building

Type	Mutable?	Thread synchronization	Typical use
StringBuffer	Yes	Synchronized	Legacy/thread-safe mutable string use cases

Example:

```
java
StringBuilder builder = new StringBuilder();

builder.append("Java");
builder.append(" ");
builder.append("Spring");
```

This avoids creating as many intermediate String objects as repeated concatenation in loops can.

Q37. Why is StringBuilder generally faster than String concatenation in a loop?

Consider:

```
java
String result = "";

for (int i = 0; i < 10000; i++) {
    result += i;
}
```

Strings are immutable.

Repeated concatenation can result in many intermediate objects.

Using:

```
java
StringBuilder
```

allows mutation of an internal character sequence.

```
java
StringBuilder result = new StringBuilder();

for (int i = 0; i < 10000; i++) {
```

```
result.append(i);  
}
```

Tricky question

Is `StringBuilder` always better than `+`?

No.

The compiler/JIT can optimize straightforward concatenation, particularly outside loops.

So the better interview answer is:

For complex or repeated string construction, especially in loops, `StringBuilder` gives explicit mutable accumulation and avoids unnecessary intermediate `String` objects.

Q38. What is autoboxing and unboxing? ☐

Autoboxing

Primitive $\rightarrow$ wrapper.

```
java  
  
int x = 10;  
  
Integer y = x;
```

Unboxing

Wrapper $\rightarrow$ primitive.

```
java  
  
Integer x = 10;  
  
int y = x;
```

Java performs the conversion automatically.

Q39. What is a Wrapper class?

Wrapper classes represent primitive values as objects.

Examples:

```
text  
  
int       $\rightarrow$  Integer
```

```
long    → Long
double  → Double
char    → Character
boolean → Boolean
```

Useful when APIs require objects.

For example:

```
java
List<Integer> numbers;
```

You cannot use:

```
java
List<int>
```

because Java generics work with reference types, not primitive types.

Q40. What is Integer caching? ☐

This is a favorite tricky question.

Consider:

```
java
Integer a = 100;
Integer b = 100;

System.out.println(a == b);
```

This can print:

```
text
true
```

because Java commonly caches a range of Integer objects, including the range `-128` to `127` by default.

But:

```
java
Integer a = 1000;
Integer b = 1000;
```

```
a == b
```

should not be relied upon to be `true`.

Correct comparison

Use:

```
java
a.equals(b)
```

for value equality.

Interview lesson

Never use `==` to compare wrapper values when you mean numeric equality.

Q41. What is an exception? ☐

An exception is an event representing an abnormal condition that disrupts normal program execution.

Example:

```
java
int result = 10 / 0;
```

produces an arithmetic exception.

Java provides exception-handling mechanisms such as:

```
java
try
catch
finally
throw
throws
```

Q42. Checked vs unchecked exceptions ☐ ☐

Checked exceptions

Checked by the compiler.

They generally extend:

```
java
Exception
```

but not:

```
java
RuntimeException
```

Example:

```
java
IOException
```

Unchecked exceptions

Usually subclasses of:

```
java
RuntimeException
```

Examples:

```
text
NullPointerException
IllegalArgumentException
IllegalStateException
```

Interview answer

Checked exceptions represent conditions the compiler requires us to account for, while unchecked exceptions generally represent programming errors or runtime conditions and don't require compile-time handling.

Q43. `throw` vs `throws` ☐

`throw`

Actually throws an exception.

```
java
throw new IllegalArgumentException("Invalid amount");
```

`throws`

Declares that a method may propagate an exception.

```
java
void readFile() throws IOException {
}
```

Simple memory trick:

```
text
throw → action
throws → declaration
```

Q44. What is finally?

`finally` is used for cleanup code that should normally execute whether an exception occurs or not.

```
java
try {
    process();
} catch (Exception e) {
    handle(e);
} finally {
    cleanup();
}
```

Important exception

`finally` does not absolutely guarantee execution in every imaginable situation, such as abrupt JVM termination.

Q45. Can finally execute without catch?

Yes.

```
java
try {
    process();
} finally {
    cleanup();
}
```

Valid Java.

Q46. What happens if both try and finally contain return statements? ☐

Example:

```
java
int test() {

    try {
        return 10;
    } finally {
        return 20;
    }

}
```

Result:

```
text
20
```

The `finally` return overrides the earlier return.

Interview warning

Never use `return` inside `finally` in normal production code because it can suppress exceptions and make control flow confusing.

Q47. What is try-with-resources? ☐

Used for automatically closing resources that implement `AutoCloseable`.

```
java
try (BufferedReader reader =
    new BufferedReader(new FileReader("data.txt"))) {

    // read

}
```

The resource is automatically closed.

This is preferred over manually managing:

```
java
finally {
    reader.close();
}
```

for supported resources.

Q48. What is Optional in Java? ☐

`Optional<T>` represents a value that may or may not be present.

Example:

```
java
Optional<User> user =
    userRepository.findById(id);
```

Instead of returning:

```
java
null
```

the method explicitly communicates possible absence.

Example:

```
java
return userRepository
    .findById(id)
    .orElseThrow(() ->
        new UserNotFoundException(id));
```

Important interview point

`Optional` is not a universal replacement for null everywhere.

It is primarily useful for expressing optional return values and improving API clarity.

Q49. What is the difference between `orElse()` and `orElseGet()`? ☐

Very important.

```
java
optional.orElse(createDefault());
```

The expression passed to `orElse()` is evaluated eagerly.

Whereas:

```
java
optional.orElseGet(() -> createDefault());
```


allows lazy evaluation.

So if creating the default value is expensive:

```
java
    orElseGet()
```

can avoid unnecessary work when the Optional already contains a value.

Q50. What is Java Pass-by-Value? ☐☐☐

This is one of the most misunderstood Java questions.

Java is always pass-by-value.

For primitives:

```
java
void change(int x) {
    x = 20;
}
```

The original value is unchanged.

For objects:

```
java
void change(User user) {
    user.setName("John");
}
```

The object's state can change because the copied value is a **reference to the same object**.

But the reference itself is passed by value.

Example

```
java
void change(User user) {
    user = new User("New");
}
```

The caller's reference does not get replaced.

Best interview answer

Java is strictly pass-by-value. For objects, the value being passed is a copy of the reference, which is why the method can modify the referenced object's state but cannot replace the caller's reference.

This answer is much better than:

"Objects are passed by reference."

That statement is technically incorrect for Java.

□ FIRST 50 — WHAT YOU SHOULD MASTER

Don't move ahead just because you have read these.

You should be able to answer these **without looking at notes**:

text

1. Java features
2. JDK/JRE/JVM
3. Platform independence
4. Why Java isn't purely OO
5. OOP principles
6. Encapsulation vs abstraction
7. Inheritance
8. Polymorphism
9. Overloading vs overriding
10. Return-type overloading
11. Static method overriding
12. Private method overriding
13. Constructor inheritance
14. this vs super
15. Abstract class
16. Interface
17. Abstract class vs interface
18. Multiple inheritance
19. Composition
20. IS-A vs HAS-A
21. Immutable class
22. Truly immutable class
23. final/finally/finalize
24. Constructor
25. Constructor chaining

```
26. Object creation
27. Stack vs heap
28. Garbage collection
29. System.gc()
30. Memory leaks
31. == vs equals
32. equals/hashCode
33. equals without hashCode
34. String pool
35. String immutability
36. StringBuilder/StringBuffer
37. String concatenation
38. Autoboxing/unboxing
39. Wrapper classes
40. Integer caching
41. Exceptions
42. Checked/unchecked
43. throw/throws
44. finally
45. finally without catch
46. return in finally
47. try-with-resources
48. Optional
49. orElse/orElseGet
50. Pass-by-value
```

□ What comes next in Module 1

The next 50 will be **Collections + Generics**, which is one of the most heavily tested areas for a 2-year Java backend engineer:

Questions 51–100

We'll go deep into:

- `ArrayList`
- `LinkedList`
- `Vector`
- `HashMap`
- `LinkedHashMap`

- `TreeMap`
- `HashSet`
- `LinkedHashSet`
- `TreeSet`
- `ConcurrentHashMap`
- `Hashtable`
- `PriorityQueue`
- `Deque`
- `ArrayDeque`
- `Comparable`
- `Comparator`
- `Iterator`
- `ConcurrentModificationException`
- fail-fast vs fail-safe
- `HashMap` internals
- collision handling
- resizing
- load factor
- capacity
- treeification
- `equals/hashCode`
- Generics
- PECS
- `? extends`
- `? super`
- type erasure

And these will include the **classic product-company traps**, such as:

Why does `HashMap` allow one null key?

What happens internally when `HashMap` crosses its threshold?

Why is `HashMap` not thread-safe?

Why can `ConcurrentHashMap` not store null keys/values?

Why is `ArrayList` usually faster for random access?

Why is `LinkedList` often a poor choice despite $O(1)$ insertion/removal at a known node?

What happens if you modify a collection while iterating?

Comparable vs Comparator?

What does ? super T actually mean?

Why can't Java generics use primitive types?

Those are the questions that start moving you from "**I know Java**" toward "**I can handle a product-company Java interview.**"

you asked: ok as above

message time: 2026-08-14 21:19:25

ok as above continues

The above is the **high-value master map**, not literally every possible interview question. Trying to dump 500+ questions into one response would make it much harder to study.

For your goal, I recommend turning this into a structured "**2-Year Java + Spring Boot Product Company Interview Bible**" with:

Module 1: 150 Core Java questions + answers **Module 2:** 120 Spring/Spring Boot questions + internal working **Module 3:** 80 JPA/Hibernate questions **Module 4:** 75 REST/API questions **Module 5:** 75 Microservices questions **Module 6:** 50 Spring Security/JWT questions **Module 7:** 50 SQL questions + queries **Module 8:** 50 Multithreading questions **Module 9:** 50 Kafka + Redis questions **Module 10:** 50 Docker/AWS questions **Module 11:** 100 DSA problems specifically for 2-year Java SDE interviews **Module 12:** 50 LLD/system-design scenarios **Module 13:** 100 project/resume follow-up questions **Module 14:** Company-specific patterns for **Amazon, Google, Microsoft, Adobe, Uber, Walmart, Mastercard, Morgan Stanley, Cisco, Salesforce, Atlassian, Oracle, etc.**

Most importantly, for every question I'd structure it as:

Question → Short interview answer → Detailed explanation → Real project example → Follow-up question → Tricky interviewer counter-question → Best answer → Code/diagram where relevant

That format will train you to **answer follow-up questions instead of just memorizing definitions.**

chatgpt response: # Module 1

Module 1 — Core Java

Questions 51–100: Collections + Generics

We continue exactly from Q1–50. This section is **very high priority** for 2-year Java/Spring Boot interviews. Questions around `HashMap`, `collections`, `equals/hashCode`, `Comparable/Comparator`,

iterators, and generics are frequent because interviewers can easily move from a basic question into internal implementation and problem-solving.

SECTION B — JAVA COLLECTIONS

Q51. What is the Java Collection Framework? □

Short interview answer

The Java Collection Framework is a set of interfaces, implementations, and algorithms used to store and manipulate groups of objects.

The major interfaces are:

```
text
Collection
├── List
├── Set
└── Queue

Map    ← separate hierarchy
```

Examples:

```
text
List
├── ArrayList
├── LinkedList
└── Vector

Set
├── HashSet
├── LinkedHashSet
└── TreeSet

Queue
├── PriorityQueue
└── Deque
    └── ArrayDeque

Map
├── HashMap
```

```
├─ LinkedHashMap
├─ TreeMap
└─ ConcurrentHashMap
```

Real project example

Suppose your Order Service needs to store:

```
java
List<Order> orders;
```

If you need unique user IDs:

```
java
Set<Long> userIds;
```

If you need user → order mapping:

```
java
Map<Long, List<Order>> ordersByUser;
```

Follow-up

Is Map a child of Collection?

No.

`Map` is part of the Java Collections Framework but does not extend `Collection`.

Q52. Collection vs Collections? ☐

This is a classic trap.

‘Collection’

An interface.

```
java
Collection<String> names;
```

It represents a group of objects.

‘Collections’

A utility class containing static methods for collection operations.

Examples:

```
java
Collections.sort(list);
Collections.reverse(list);
Collections.max(list);
Collections.unmodifiableList(list);
```

Interview answer

Collection is an interface representing a group of objects, whereas Collections is a utility class providing static helper methods for working with collections.

Q53. Collection vs Map? ☐

Collection

Stores individual elements.

```
java
List<String>
Set<String>
```

Map

Stores key-value pairs.

```
java
Map<Long, User>
```

Example:

```
text
Collection:

[A, B, C, D]

Map:

1 → User1
2 → User2
3 → User3
```

Important:

Map does not extend Collection.

Q54. List vs Set vs Map? ☐ ☐

Type	Duplicate elements	Ordering	Key-value
List	Allowed	Usually maintained	No
Set	Not allowed according to set equality semantics	Depends implementation	No
Map	Keys unique	Depends implementation	Yes

Examples:

```
java
List<String> list = new ArrayList<>();

Set<String> set = new HashSet<>();

Map<Long, String> map = new HashMap<>();
```

Real-world selection

If order matters and duplicates are allowed:

```
java
List
```

If uniqueness matters:

```
java
Set
```

If lookup by key is needed:

```
java
Map
```

Q55. ArrayList vs LinkedList ☐ ☐ ☐

This is extremely common.

ArrayList

Internally uses a dynamically resizable array.

```
text
```

```
[10] [20] [30] [40] [50]
```

Advantages:

- fast random access
- cache-friendly contiguous storage characteristics
- usually the default choice for list workloads

Access:

```
java
```

```
list.get(100);
```

is approximately:

```
text
```

```
 $O(1)$ 
```

LinkedList

Uses linked nodes.

Conceptually:

```
text
```

```
Node → Node → Node → Node
```

Each node contains links to neighboring nodes.

Random access:

```
java
```

```
list.get(100);
```

requires traversal and is approximately:

```
text
```

```
 $O(n)$ 
```

Critical interview point

People often say:

| *"LinkedList is faster for insertion."*

That answer is incomplete.

Insertion/removal can be **O(1)** once you already have the relevant node/position, but finding that position can itself be O(n).

Therefore, in real applications, `ArrayList` is often preferable unless you have a specific reason to use a linked structure.

Q56. Why is ArrayList access O(1)?

Because elements are stored in an array-like structure.

If:

```
text
index = 5
```

the runtime can directly calculate the location corresponding to that index.

Conceptually:

```
text
base address + index × element size
```

Therefore:

```
java
list.get(5);
```

doesn't need to traverse elements $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$.

Q57. What happens when ArrayList capacity is exceeded? ☐

Suppose the internal array has insufficient capacity.

When another element needs to be inserted:

```
text
Old array
[ A ][ B ][ C ][ D ]

      ↓ resize

New larger array
[ A ][ B ][ C ][ D ][ ][ ][ ]
```

Existing elements are copied/moved into the larger backing storage.

Therefore, a resize operation can be expensive.

Why is `add()` still considered amortized $O(1)$?

Because resizing doesn't happen for every insertion.

Most insertions are cheap; occasional resizing is expensive.

Over many insertions, the average cost remains amortized $O(1)$.

Product-company follow-up

Can we reduce resizing overhead?

Yes.

If you know approximately how many elements you'll store:

```
java
List<User> users = new ArrayList<>(10000);
```

you can provide an initial capacity.

Q58. ArrayList size vs capacity? ☐

These are different.

Size

Number of actual elements.

```
text
size = 5
```

Capacity

Amount of backing-array storage currently available before resizing is required.

You can have conceptually:

```
text
size = 5
capacity = 10
```

The public `ArrayList` API exposes `size()`, but capacity is an implementation detail rather than a normal public property.

Q59. Why is LinkedList insertion considered $O(1)$?

Suppose you already have a reference to the node where insertion should occur.

You can change the links:

```
text
A → B
A → X → B
```

That is constant-time pointer/reference manipulation.

But:

```
java
LinkedList.add(500000, value);
```

may first need to traverse to that position.

So the full operation can involve:

```
text
find position → O(n)
insert → O(1)
```

Best answer

Linked-list insertion/removal is $O(1)$ when the relevant node/iterator position is already known; locating an arbitrary index is $O(n)$.

Q60. ArrayList vs LinkedList in a real backend project? ☐

Suppose you're building:

```
text
OrderService
```

and need:

```
java
List<Order> orders;
```

I'd normally choose:

```
java
ArrayList
```

unless there's a specific workload requiring linked-node semantics.

Why?

- random access is fast
- better memory locality
- lower per-element overhead
- generally performs better for common list operations

Strong product-company answer

I wouldn't choose LinkedList simply because insertion is theoretically $O(1)$. I'd look at the actual access pattern. For most application-level list workloads, ArrayList is the better default.

Q61. HashSet vs ArrayList? ☐

ArrayList

Allows duplicates.

```
text
[A, B, A, C]
```

HashSet

Maintains unique elements according to `equals/hashCode`.

```
text
[A, B, C]
```

Lookup

`HashSet` provides expected $O(1)$ average-time lookup.

`ArrayList.contains()` is $O(n)$.

Example:

```
java
Set<String> emails = new HashSet<>();

emails.add("a@gmail.com");
emails.add("a@gmail.com");
```

The duplicate is not added.

Q62. How does HashSet work internally? ☐ ☐

This is a favorite follow-up.

`HashSet` is backed by a `HashMap`.

Conceptually:

```
text
  HashSet
    ↓
  HashMap
    ↓
hash table
```

When you do:

```
java
set.add(user);
```

the set uses the object's hash code to determine where the element should be stored.

Equality is then used to determine whether an equivalent element already exists.

Important

This is why correct:

```
java
equals()
hashCode()
```

implementations matter.

Q63. `HashMap` internal working? ☐ ☐ ☐

This is one of the **must-master questions**.

Suppose:

```
java
Map<String, User> users = new HashMap<>();

users.put("john", user);
```

Conceptually:

```
text
"john"
  ↓
hashCode()
  ↓
hash transformation
  ↓
bucket index
  ↓
entry/node
```

The HashMap stores the key-value mapping in a hash table.

If multiple keys map to the same bucket, that's a collision.

Modern Java implementations can use a tree structure for heavily collided buckets under certain conditions.

Simplified structure

```
text
HashMap

Bucket 0 → null
Bucket 1 → Node
Bucket 2 → Node → Node
Bucket 3 → null
Bucket 4 → Node
...
```

Q64. What is a hash collision?

A collision occurs when multiple keys end up in the same bucket.

For example:

```
text
Key A → bucket 5
Key B → bucket 5
```

Both need to coexist.

HashMap handles this through its collision-resolution mechanism.

Important distinction

Two different objects can have the same hash code.

That is allowed.

But if:

```
java
a.equals(b)
```

is true, they **must** have the same hash code.

Q65. What happens when two keys have the same hash code? ☐

HashMap places them in the same bucket and then uses equality comparison to distinguish them.

Conceptually:

```
text
Bucket 5

Node(keyA, valueA)
    ↓
Node(keyB, valueB)
```

When looking up a key:

1. calculate hash
2. identify bucket
3. compare hash values
4. use `equals()` to identify the correct key

This is why:

Hash code identifies a candidate bucket; equals() determines logical key equality.

Q66. What is HashMap load factor? ☐ ☐

Load factor determines how full a HashMap can become before resizing.

The commonly used default load factor is:

```
text
0.75
```

Conceptually:

```
text
threshold = capacity × load factor
```

If:

```
text
capacity = 16
load factor = 0.75
```

then:

```
text
threshold = 12
```

When the map grows beyond its threshold, resizing occurs.

Why 0.75?

It's a tradeoff between:

- memory usage
- collision frequency
- lookup performance

Q67. What happens when HashMap is resized? ☐

When the map crosses its threshold, the table is resized.

Modern Java implementations generally grow the table, commonly doubling its capacity.

Entries are redistributed according to the new table size.

Conceptually:

```
text
Capacity 16
    ↓
Capacity 32
```

This operation is more expensive than a normal insertion.

Important

HashMap's average lookup/insert complexity remains expected $O(1)$, despite occasional resizing.

Q68. What is the initial capacity of HashMap?

The standard constructor behavior and actual table allocation details are important.

A common interview trap is saying:

"HashMap immediately creates an array of size 16 when you call new HashMap()."

Modern Java implementations can defer allocation until the first insertion.

The commonly associated default initial capacity is:

```
text
16
```

with default load factor:

```
text
0.75
```

but actual internal allocation timing matters.

Strong answer

The conventional default initial capacity is 16 and default load factor is 0.75, but modern HashMap can defer allocating its internal table until the first insertion.

Q69. Why should HashMap keys be immutable? ☐ ☐

Consider:

```
java
Map<User, String> map = new HashMap<>();
```

Suppose `User` fields used in:

```
java
hashCode()
equals()
```

are changed after insertion.

Then the key might conceptually move from:

```
text
Bucket 3
```

to logically belonging to:

text

Bucket 10

but the HashMap still stores it in its original bucket.

Now lookup may fail.

Example concept

java

```
User user = new User(10);

map.put(user, "John");

user.setId(20);

map.get(user);
```

Depending on how equality/hashCode are defined, this can fail unexpectedly.

Best practice

Use immutable key state.

Commonly safe keys:

text

String
Integer
Long
UUID
immutable value objects

Q70. Can HashMap have null keys and values? ☐

Yes.

A HashMap allows:

text

one null key
multiple null values

Example:

```

java
Map<String, String> map = new HashMap<>();

map.put(null, "value");
map.put("A", null);
map.put("B", null);

```

Compare with ConcurrentHashMap

ConcurrentHashMap does **not** permit null keys or null values.

We'll discuss why shortly.

Q71. HashMap vs Hashtable □

HashMap	Hashtable
Not synchronized by default	Synchronized
Allows null key/value	Does not allow null key/value
Modern/common choice	Legacy
Better general-purpose performance	More synchronization overhead
Supports modern Collections APIs	Legacy class

Important

Don't say:

Hashtable is simply a thread-safe HashMap.

It's a legacy synchronized map with different characteristics.

For concurrent applications, consider:

```

java
ConcurrentHashMap

```

rather than automatically using Hashtable.

Q72. HashMap vs ConcurrentHashMap □ □ □

HashMap

Not thread-safe for concurrent structural modification.

ConcurrentHashMap

Designed for concurrent access.

It provides thread-safe operations with substantially better concurrency characteristics than synchronizing the entire map.

Example:

```
java
Map<String, User> users =
    new ConcurrentHashMap<>();
```

Why not just synchronize HashMap?

You can wrap maps with synchronization mechanisms, but that can serialize access more broadly.

`ConcurrentHashMap` is specifically designed for concurrent access patterns.

Important property

`ConcurrentHashMap` does not allow:

```
java
null key
null value
```

Why?

One important reason is that in concurrent operations, `null` cannot reliably distinguish:

```
text
key absent
```

from:

```
text
key present with null value
```

The API therefore disallows nulls to avoid ambiguity.

Q73. What is LinkedHashMap? ☐

`LinkedHashMap` maintains a predictable iteration order.

By default, that is insertion order.

Example:

```

java
Map<Integer, String> map =
    new LinkedHashMap<>();

map.put(3, "C");
map.put(1, "A");
map.put(2, "B");

```

Iteration:

```

text
3
1
2

```

Important feature

It can also be configured for **access order**, which makes it useful for implementing LRU-style caches.

Example:

```

java
new LinkedHashMap<>(16, 0.75f, true);

```

The final `true` enables access-order mode.

Q74. What is TreeMap? □

`TreeMap` stores keys in sorted order according to their natural ordering or a supplied `Comparator`.

Example:

```

java
Map<Integer, String> map =
    new TreeMap<>();

map.put(30, "C");
map.put(10, "A");
map.put(20, "B");

```

Iteration:

```
text
```

```
10
```

```
20
```

```
30
```

Typical complexity:

```
text
```

```
get      → O(log n)
```

```
put      → O(log n)
```

```
remove → O(log n)
```

because it is based on a balanced tree structure.

Q75. HashMap vs LinkedHashMap vs TreeMap □ □

Feature	HashMap	LinkedHashMap	TreeMap
Ordering	No guaranteed iteration order	Insertion/access order	Sorted
Average lookup	O(1)	O(1)	O(log n)
Null key	Yes	Yes	Depends on ordering/comparator; natural ordering generally doesn't permit null
Use case	Fast lookup	Ordered lookup / LRU-style	Sorted keys/range operations

Practical selection

Need fast key lookup:

```
text
```

```
HashMap
```

Need insertion/access order:

```
text
```

```
LinkedHashMap
```

Need sorted keys:

```
text
```

```
TreeMap
```


Q76. HashSet vs LinkedHashSet vs TreeSet □

Feature	HashSet	LinkedHashSet	TreeSet
Duplicates	No	No	No
Ordering	No guaranteed order	Insertion order	Sorted
Typical lookup	O(1) expected	O(1) expected	O(log n)
Backing concept	Hash table	Hash table + links	Balanced tree

Example

If you need:

unique products in the order users selected them

Use:

```
java
LinkedHashSet<Product>
```

If you need:

unique products sorted by product ID

Use:

```
java
TreeSet<Product>
```

Q77. What is Comparable? □ □

`Comparable<T>` defines the object's **natural ordering**.

Example:

```
java
class Employee implements Comparable<Employee> {

    private int salary;

    @Override
    public int compareTo(Employee other) {
        return Integer.compare(this.salary, other.salary);
    }
}
```

```
}  
}
```

Then:

```
java  
Collections.sort(employees);
```

can use that natural ordering.

Interview phrase

Comparable defines the default/natural ordering of a type.

Q78. What is Comparator? □ □

`Comparator<T>` defines an external/custom ordering.

Example:

```
java  
  
Comparator<Employee> bySalary =  
    Comparator.comparing(Employee::getSalary);
```

Another:

```
java  
  
Comparator<Employee> byName =  
    Comparator.comparing(Employee::getName);
```

Now the same `Employee` type can be sorted differently.

Q79. Comparable vs Comparator □ □ □

Comparable	Comparator
Defines natural ordering	Defines external/custom ordering
<code>compareTo()</code>	<code>compare()</code>
Implemented by class	Separate object
Usually one natural order	Can define many orders

Example

Employee can naturally be ordered by:

```
text
employeeId
```

but different reports might need:

```
text
salary
name
joiningDate
```

Use `Comparator` for those alternate orderings.

Q80. What does ``compareTo()`` return? ☐

It should indicate ordering:

```
text
negative → this < other
zero      → equal in ordering
positive → this > other
```

Don't rely on exact values such as:

```
text
-1
0
1
```

A comparator can return any negative/zero/positive integer.

Better:

```
java
return Integer.compare(this.age, other.age);
```

instead of:

```
java
return this.age - other.age;
```

because subtraction can overflow.

Q81. Why can ``a - b`` be dangerous in `Comparator`? ☐

Consider:

```
java
Comparator<Integer> c =
    (a, b) -> a - b;
```

For extreme integer values, subtraction can overflow.

Example concept:

```
text
Integer.MAX_VALUE
-
(-1)
```

can overflow.

Prefer:

```
java
Integer.compare(a, b);
```

or:

```
java
Comparator.comparingInt(...)
```

Q82. What is PriorityQueue? ☐

`PriorityQueue` is a queue where elements are ordered according to priority rather than insertion order.

By default, the smallest element has highest priority according to natural ordering.

```
java
PriorityQueue<Integer> queue =
    new PriorityQueue<>();

queue.add(30);
queue.add(10);
queue.add(20);
```

Calling:

```
java
queue.poll();
```

returns:

```
text
10
```

Typical implementation

A heap-based structure.

Complexity

Typical:

```
text
offer → O(log n)
poll  → O(log n)
peek  → O(1)
```

Q83. Does iterating over PriorityQueue give sorted order? ☐

No.

This is a common trap.

The queue guarantees that the **head** has the highest priority.

Its iterator does not guarantee sorted traversal.

If you need repeated sorted removal:

```
java
while (!queue.isEmpty()) {
    System.out.println(queue.poll());
}
```

Q84. What is Deque?

Deque means:

Double Ended Queue.

Elements can be inserted/removed from both ends.

text

```
Front ← [ A ][ B ][ C ] → Rear
```

Java provides:

java

```
Deque<Integer> deque =  
    new ArrayDeque<>();
```

Useful operations:

java

```
addFirst()  
addLast()  
  
removeFirst()  
removeLast()  
  
peekFirst()  
peekLast()
```

It can implement both:

text

```
Stack  
Queue
```

Q85. ArrayDeque vs Stack ☐

Modern Java code generally prefers:

java

```
Deque<Integer> stack =  
    new ArrayDeque<>();
```

over the legacy:

java

```
Stack<Integer>
```

because `ArrayDeque` is designed as a modern double-ended queue and generally has better API/design characteristics for stack/queue use cases.

Example:

```
java
stack.push(10);
stack.push(20);

stack.pop();
```

Q86. What is Iterator? ☐

`Iterator` provides a standard way to traverse collections.

```
java
Iterator<String> iterator =
    names.iterator();

while (iterator.hasNext()) {
    String name = iterator.next();
}
```

Methods include:

```
text
hasNext()
next()
remove()
```

where supported.

Q87. Why should we use Iterator instead of modifying a collection inside a for-each loop? ☐

This can cause:

```
text
ConcurrentModificationException
```

Example:

```
java
for (String name : names) {
    if (name.equals("John")) {
        names.remove(name);
    }
}
```

```
}  
}
```

This can fail.

Instead:

```
java  
Iterator<String> iterator = names.iterator();  
  
while (iterator.hasNext()) {  
    String name = iterator.next();  
  
    if (name.equals("John")) {  
        iterator.remove();  
    }  
}
```

The iterator's removal operation is designed for this use.

Q88. What is ConcurrentModificationException? ☐ ☐

It commonly occurs when a collection is structurally modified while being iterated using an iterator that detects such modification.

Example:

```
java  
for (String value : list) {  
    list.remove(value);  
}
```

Potential result:

```
text  
ConcurrentModificationException
```

Important trap

Despite its name, it does **not necessarily mean multiple threads are involved**.

It can happen in a single-threaded program.

Q89. What is fail-fast behavior?

A fail-fast iterator detects certain structural modifications outside the iterator's supported operations and throws an exception rather than continuing with potentially inconsistent traversal.

Examples commonly discussed:

```
text
ArrayList
HashMap
HashSet
```

Important

Fail-fast behavior is a best-effort safety mechanism, not a synchronization guarantee.

Q90. Is fail-fast thread-safe? ☐

No.

Fail-fast behavior does not make a collection thread-safe.

If multiple threads need to modify/access shared mutable collections, use appropriate concurrency mechanisms.

Q91. What is CopyOnWriteArrayList? ☐

`CopyOnWriteArrayList` is a thread-safe list implementation designed for workloads where:

Reads are frequent, writes are relatively rare.

On modification, it creates a new underlying array.

Conceptually:

```
text
Read:
same array

Write:
create new copy
    ↓
modify copy
    ↓
publish new array
```

Good use case

Configuration/listener lists where:

```
text
reads >> writes
```

Bad use case

Thousands of writes per second.

Because each write can require copying the array.

Q92. What is ConcurrentHashMap? ☐ ☐

A thread-safe map designed for concurrent access.

Example:

```
java
ConcurrentHashMap<String, Integer> counts =
    new ConcurrentHashMap<>();
```

Useful when multiple threads need to read/update shared mappings concurrently.

Modern implementations use sophisticated concurrency mechanisms rather than simply synchronizing every operation on the whole map.

Q93. What is `putIfAbsent()`? ☐

It atomically inserts a mapping only if the key is not already present.

```
java
map.putIfAbsent("A", 100);
```

This is especially useful in concurrent code because doing:

```
java
if (!map.containsKey(key)) {
    map.put(key, value);
}
```

is not an atomic check-then-act operation.

Q94. Why is `containsKey()` + `put()` dangerous in concurrent code?

Suppose:

```
java
if (!map.containsKey(key)) {
    map.put(key, value);
}
```

Two threads can execute:

```
text
Thread 1 → containsKey → false
Thread 2 → containsKey → false

Thread 1 → put
Thread 2 → put
```

The check and insertion are separate operations.

Use atomic methods where appropriate:

```
java
putIfAbsent()
computeIfAbsent()
compute()
merge()
```

with a concurrent map.

Q95. What is `computeIfAbsent()`? ☐

Useful when you want to create a value only if the key is absent.

Example:

```
java
Map<String, List<Order>> ordersByUser =
    new HashMap<>();

ordersByUser
    .computeIfAbsent(userId, k -> new ArrayList<>())
    .add(order);
```

Instead of:

```

java
if (!map.containsKey(userId)) {
    map.put(userId, new ArrayList<>());
}

map.get(userId).add(order);

```

It makes the intent clearer and can be especially useful with concurrent maps for atomic map-level operations.

Q96. What is the difference between `remove()` and `poll()` in Queue? ★

Depending on the queue API:

`remove()`

Removes and returns the head.

If empty:

```

text
NoSuchElementException

```

`poll()`

Removes and returns the head.

If empty:

```

text
null

```

Similarly:

```

text
element() → exception if empty
peek()    → null if empty

```

Memory trick

```

text
poll / peek → safer empty behavior
remove / element → exception on empty

```

Q97. What is an unmodifiable collection? □

An unmodifiable collection prevents modification through the returned view/reference.

Example:

```
java
List<String> original =
    new ArrayList<>();

List<String> readOnly =
    Collections.unmodifiableList(original);
```

Calling:

```
java
readOnly.add("A");
```

throws an exception.

Modern Java also provides factory methods such as:

```
java
List.of("A", "B", "C");
```

which create immutable/unmodifiable collection instances.

Important distinction

An unmodifiable **view** is not necessarily an immutable copy of the underlying collection.

If the original collection changes, the view can reflect those changes.

Q98. What is Generics? □ □

Generics provide compile-time type safety and allow classes/methods to work with different types without unnecessary casts.

Without generics:

```
java
List list = new ArrayList();

list.add("Java");

list.add(10);
```

Potential type problems occur at runtime.

With generics:

```
java
List<String> list = new ArrayList<>();

list.add("Java");
```

Now:

```
java
list.add(10);
```

is rejected at compile time.

Benefits

- type safety
- fewer casts
- reusable code
- clearer APIs

Q99. Why can't we use primitive types with Generics? ☐

This is invalid:

```
java
List<int> numbers;
```

because Java generics operate on reference types.

Use:

```
java
List<Integer> numbers;
```

Java performs autoboxing/unboxing when appropriate.

Why was it designed this way?

Java generics are implemented primarily through **type erasure**, rather than generating separate generic implementations for every primitive type.

Q100. What is Type Erasure? ☐☐☐

This is one of the most important advanced generic questions.

Java generics provide type checking at compile time, but generic type information is largely erased from the runtime representation.

Example:

```
java
List<String>
```

and:

```
java
List<Integer>
```

are generally represented as the same raw runtime class:

```
text
java.util.ArrayList
```

The compiler inserts necessary casts/checks.

Why does Java use type erasure?

Primarily to maintain compatibility with older Java code and the existing JVM/runtime model.

Important consequence

You cannot generally do:

```
java
if (obj instanceof List<String>) {
}
```

because the parameterized type isn't available in that form at runtime.

You can do:

```
java
if (obj instanceof List<?>) {
}
```

☐ The 50 Questions You Now Need to Master

You have now completed:

```
text
Q1-50
Core Java fundamentals
```

OOP
String
Exceptions
Memory
Object model

Q51-100
Collections
HashMap
HashSet
ArrayList
LinkedList
TreeMap
LinkedHashMap
TreeSet
PriorityQueue
Deque
Iterator
Concurrency collections
Generics
Type erasure

The **highest-priority questions from Q51–100** are:

text

- ☐ Q55 ArrayList vs LinkedList
- ☐ Q57 ArrayList resizing
- ☐ Q62 HashSet internals
- ☐ Q63 HashMap internals
- ☐ Q65 Hash collisions
- ☐ Q66 Load factor
- ☐ Q67 HashMap resizing
- ☐ Q69 Mutable HashMap keys
- ☐ Q72 HashMap vs ConcurrentHashMap
- ☐ Q75 HashMap vs LinkedHashMap vs TreeMap
- ☐ Q76 HashSet vs LinkedHashSet vs TreeSet
- ☐ Q77 Comparable
- ☐ Q78 Comparator
- ☐ Q79 Comparable vs Comparator

- Q87 Iterator modification
- Q88 ConcurrentModificationException
- Q92 ConcurrentHashMap
- Q93 putIfAbsent
- Q94 atomic check-then-act
- Q98 Generics
- Q100 Type Erasure

One especially important interview chain

If the interviewer asks:

"How does HashMap work?"

don't stop after:

"It uses hashing."

Your answer should be capable of continuing:

text

```
HashMap
↓
hashCode()
↓
hash spreading/transformation
↓
bucket index
↓
collision
↓
equals()
↓
linked/tree bucket structure
↓
load factor
↓
threshold
↓
resize
↓
capacity
```

```
↓  
mutable-key problem  
  
↓  
thread safety  
  
↓  
ConcurrentHashMap
```

If you can confidently explain that entire chain, you're already much better prepared for a typical 2-year Java backend interview.

Next: Module 1 — Q101–150

The final 50 Core Java questions will move into **Java 8+, Functional Programming, Streams, JVM, GC, Reflection, Design Patterns, and interview coding traps**.

The especially important topics will be:

- Lambda expressions
- Functional interfaces
- Predicate, Function, Consumer, Supplier
- Method references
- Stream API
- map() vs flatMap()
- filter()
- reduce()
- collect()
- groupingBy()
- partitioningBy()
- intermediate vs terminal operations
- lazy evaluation
- parallel streams
- Optional
- default/static interface methods
- JVM architecture
- class loading
- JIT
- heap generations
- GC concepts
- strong/weak/soft references

- reflection
- annotations
- records
- sealed classes
- common Java coding questions

After **Q150**, we'll move directly into **Module 2 — 120 Spring/Spring Boot questions**, where we'll go much deeper than the initial Spring overview: **IoC internals, BeanFactory/ApplicationContext, bean lifecycle, auto-configuration internals, conditional beans, proxies, AOP, transactions, Spring MVC internals, configuration, profiles, Actuator, testing and production scenarios.**

The HashMap chain is central to collection interviews, and its concurrency boundary becomes clearer when you in backend code.

you asked: ok as above

message time: 2026-08-14 21:30:39

ok as above continues

The above is the **high-value master map**, not literally every possible interview question. Trying to dump 500+ questions into one response would make it much harder to study.

For your goal, I recommend turning this into a structured “**2-Year Java + Spring Boot Product Company Interview Bible**” with:

Module 1: 150 Core Java questions + answers **Module 2:** 120 Spring/Spring Boot questions + internal working **Module 3:** 80 JPA/Hibernate questions **Module 4:** 75 REST/API questions **Module 5:** 75 Microservices questions **Module 6:** 50 Spring Security/JWT questions **Module 7:** 50 SQL questions + queries **Module 8:** 50 Multithreading questions **Module 9:** 50 Kafka + Redis questions **Module 10:** 50 Docker/AWS questions **Module 11:** 100 DSA problems specifically for 2-year Java SDE interviews **Module 12:** 50 LLD/system-design scenarios **Module 13:** 100 project/resume follow-up questions **Module 14:** Company-specific patterns for **Amazon, Google, Microsoft, Adobe, Uber, Walmart, Mastercard, Morgan Stanley, Cisco, Salesforce, Atlassian, Oracle, etc.**

Most importantly, for every question I'd structure it as:

Question → Short interview answer → Detailed explanation → Real project example → Follow-up question → Tricky interviewer counter-question → Best answer → Code/diagram where relevant

That format will train you to **answer follow-up questions instead of just memorizing definitions.**

chatgpt response: Absolutely

Absolutely. We'll continue the **same Interview Bible**, but I'm going to tighten the approach based on current 2026 interview material rather than just producing textbook questions.

I checked recent 2026 Java/Spring Boot interview material, current collections interview guides, recent interview-experience reports, and current Spring Boot question sets. The pattern is consistent: at ~2 years, interviewers increasingly ask “**how does it work?**”, “**why did you choose it?**”, and “**what happens in production?**”, not just annotation definitions. Recent reports also show Streams, HashMap internals, `equals()`, concurrency, `CompletableFuture`, `@ControllerAdvice`, Redis, and practical Spring Boot coding appearing together in technical rounds. ([CodeBegun](#))

MODULE 1 — CORE JAVA

Q101–150: Java 8+, Streams, Optional, JVM & Advanced Java

These are **very high-value questions** for your target.

SECTION C — JAVA 8+ AND FUNCTIONAL PROGRAMMING

Q101. What is a functional interface? □□□

Short interview answer

A functional interface is an interface containing **exactly one abstract method**.

It can have any number of:

- default methods
- static methods

Example:

```
java
@FunctionalInterface
public interface Calculator {
    int calculate(int a, int b);
}
```

Then:

```
java
Calculator addition = (a, b) -> a + b;

System.out.println(addition.calculate(10, 20));
```

Output:

```
text
30
```

Why is it important?

Functional interfaces are the foundation for:

```
text
Lambda expressions
Streams
Method references
Functional programming
```

Common built-in functional interfaces

Interface	Input	Output
Predicate<T>	T	boolean
Function<T,R>	T	R
Consumer<T>	T	void
Supplier<T>	none	T

Follow-up

Can a functional interface have default methods?

Yes.

```
java
@FunctionalInterface
interface Test {
    void execute();

    default void log() {
        System.out.println("Logging");
    }
}
```

It still has only one **abstract** method.

Q102. What is `@FunctionalInterface`?

It is an annotation that tells the compiler:

This interface is intended to contain exactly one abstract method.

Example:

```
java
@FunctionalInterface
interface PaymentProcessor {
    void process();
}
```

If someone later adds another abstract method:

```
java
void refund();
```

the compiler reports an error.

Interview point

The annotation itself isn't what makes the interface functional.

An interface is functionally usable if it has one abstract method; the annotation mainly provides compile-time verification and communicates intent.

Q103. What are Lambda expressions? ☐

A lambda is a concise way of representing behavior/function implementation.

Without lambda:

```
java
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println("Running");
    }
};
```

With lambda:

```
java
Runnable r = () ->
    System.out.println("Running");
```

Another:

```
java
Comparator<Integer> comparator =
    (a, b) -> Integer.compare(a, b);
```

Interview answer

Lambda expressions allow us to pass behavior as data and are mainly used with functional interfaces.

Q104. What is the difference between Lambda and Anonymous Inner Class?

Lambda

```
java
Runnable r = () -> System.out.println("Hello");
```

Anonymous class

```
java
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println("Hello");
    }
};
```

Lambda:

- shorter syntax
- intended for functional interfaces
- has different `this` semantics
- doesn't create a separate anonymous-class declaration in source form

Important follow-up

Inside an anonymous class:

```
java
this
```

refers to the anonymous class instance.

Inside a lambda:

```
java
this
```

refers to the enclosing object.

That's a common tricky question.

Q105. What is Predicate? ☐

`Predicate<T>` represents a function that:

```
text
T → boolean
```

Example:

```
java
Predicate<Integer> isEven =
    n -> n % 2 == 0;
```

Usage:

```
java
System.out.println(isEven.test(10));
```

Output:

```
text
true
```

Very common in Streams:

```
java
numbers.stream()
    .filter(n -> n % 2 == 0)
    .toList();
```

Q106. What is Function?

`Function<T, R>` accepts one value and returns another.

```
java
Function<String, Integer> length =
    str -> str.length();
```

Usage:

```
java
length.apply("Java");
```


Result:

```
text
4
```

Conceptually:

```
text
String → Integer
```

Q107. What is Consumer?

`Consumer<T>` accepts a value and returns nothing.

```
java
Consumer<String> printer =
    value -> System.out.println(value);
```

Usage:

```
java
printer.accept("Java");
```

Common Stream usage:

```
java
list.forEach(System.out::println);
```

Q108. What is Supplier?

`Supplier<T>` doesn't accept an input but returns a value.

```
java
Supplier<Double> random =
    () -> Math.random();
```

Usage:

```
java
Double value = random.get();
```

Conceptually:

text

nothing → T

Q109. Predicate vs Function vs Consumer vs Supplier □

Memorize this table:

Interface	Input	Output	Method
Predicate<T>	T	boolean	test()
Function<T,R>	T	R	apply()
Consumer<T>	T	void	accept()
Supplier<T>	none	T	get()

Interview shortcut

text

Predicate → question
Function → transformation
Consumer → action
Supplier → generation

Q110. What is a Method Reference? □

Method references provide shorter syntax for certain lambdas.

Instead of:

java

```
list.forEach(name ->
    System.out.println(name));
```

we can write:

java

```
list.forEach(System.out::println);
```

Types include:

text

Class::staticMethod
object::instanceMethod

```
Class::instanceMethod  
Class::new
```

Example:

```
java  
  
List<String> names =  
    List.of("A", "B", "C");  
  
names.forEach(System.out::println);
```

Q111. Lambda vs Method Reference?

Lambda:

```
java  
  
x -> System.out.println(x)
```

Method reference:

```
java  
  
System.out::println
```

Both can represent the same behavior when the target functional-interface signature matches.

Interview answer

A method reference is syntactic shorthand for a lambda that simply delegates to an existing method or constructor.

SECTION D — STREAM API

Q112. What is Stream API? □□□

Streams provide a declarative way to process sequences of data.

Example:

```
java  
  
List<Integer> numbers =  
    List.of(10, 15, 20, 25, 30);  
  
List<Integer> result =  
    numbers.stream()
```

```
.filter(n -> n > 20)
.toList();
```

Result:

```
text
[25, 30]
```

Important

A Stream is **not a data structure**.

It doesn't store the data itself.

Instead, it represents a pipeline for processing data from a source.

Sources can include:

```
text
Collection
Array
I/O-related APIs
Generated streams
```

Q113. Stream vs Collection ☐

Collection

Stores data.

```
text
List → [A, B, C]
```

Stream

Processes data.

```
text
[A, B, C]
  ↓
filter
  ↓
map
  ↓
collect
```

Critical difference

A collection can generally be traversed multiple times.

A Stream is generally **single-use**.

Example:

```
java
Stream<String> stream =
    names.stream();

stream.count();

// Don't reuse the same stream here.
```

Q114. What are intermediate and terminal operations? ☐ ☐

Intermediate operations

Return another Stream.

Examples:

```
java
filter()
map()
flatMap()
sorted()
distinct()
limit()
skip()
```

Terminal operations

Produce a result or side effect.

Examples:

```
java
collect()
forEach()
count()
reduce()
```

```
min()
max()
findFirst()
anyMatch()
```

Example:

```
java
names.stream()
    .filter(name -> name.startsWith("A"))
    .map(String::toUpperCase)
    .toList();
```

Here:

```
text
filter → intermediate
map    → intermediate
toList → terminal
```

Q115. What is lazy evaluation in Streams? ☐ ☐

Intermediate operations are generally lazy.

Consider:

```
java
numbers.stream()
    .filter(n -> {
        System.out.println(n);
        return n > 10;
    });
```

Nothing is necessarily processed yet because there is no terminal operation.

Add:

```
java
    .count();
```

and the pipeline executes.

Interview answer

Stream pipelines are lazy because intermediate operations don't execute until a terminal operation triggers traversal.

Q116. Why is Stream lazy?

Lazy evaluation allows the stream implementation to:

- avoid unnecessary work
- combine operations
- stop early for short-circuiting operations
- process elements as needed

Example:

```
java
numbers.stream()
    .filter(n -> n > 100)
    .findFirst();
```

It doesn't necessarily process the entire stream once a matching element is found.

Q117. `map()` vs `flatMap()` ☐☐☐

One of the **most frequently asked Stream questions**.

`map()`

Transforms each element into one element.

```
java
List<String> names =
    List.of("John", "David");

List<Integer> lengths =
    names.stream()
        .map(String::length)
        .toList();
```

Result:

```
text
[4, 5]
```

Conceptually:

text

String → Integer

flatMap()

Transforms each element into a stream and then flattens the resulting streams.

Example:

java

```
List<List<Integer>> numbers =  
    List.of(  
        List.of(1, 2),  
        List.of(3, 4)  
    );
```

Using `map()`:

java

```
numbers.stream()  
    .map(List::stream)
```

produces conceptually:

text

Stream<Stream<Integer>>

Using `flatMap()`:

java

```
numbers.stream()  
    .flatMap(List::stream)  
    .toList();
```

produces:

text

[1, 2, 3, 4]

Memory trick

text

map → transform

flatMap → transform + flatten

Q118. Write code to flatten nested lists.

Question

Given:

```
java
List<List<String>> names =
    List.of(
        List.of("A", "B"),
        List.of("C", "D")
    );
```

Return:

```
text
[A, B, C, D]
```

Answer

```
java
List<String> result =
    names.stream()
        .flatMap(List::stream)
        .toList();
```

Interview follow-up

Why not map?

Because `map(List::stream)` produces:

```
text
Stream<Stream<String>>
```

rather than:

```
text
Stream<String>
```

Q119. What does `filter()` do?

`filter()` keeps elements satisfying a condition.

```
java
List<Integer> result =
    numbers.stream()
        .filter(n -> n % 2 == 0)
        .toList();
```

Input:

```
text
[1,2,3,4,5,6]
```

Output:

```
text
[2,4,6]
```

The parameter is effectively a:

```
text
Predicate<T>
```

Q120. What does `reduce()` do? ☐

`reduce()` combines multiple elements into one result.

Example:

```
java
int sum =
    numbers.stream()
        .reduce(0, Integer::sum);
```

Input:

```
text
[1,2,3,4]
```

Output:

```
text
10
```

Conceptually:

text

1 + 2 + 3 + 4

↓

10

Common uses

- sum
- multiplication
- maximum/minimum
- combining values

Q121. `reduce()` vs `collect()` ☐☐

reduce()

Used when you conceptually want to reduce many values into one result.

```
java
int sum = numbers.stream()
    .reduce(0, Integer::sum);
```

collect()

Used for mutable accumulation into a result container.

```
java
List<String> result =
    names.stream()
        .collect(Collectors.toList());
```

or modern Java:

```
java
List<String> result =
    names.stream().toList();
```

Interview answer

reduce is primarily for immutable-style reduction into a single value, whereas collect is designed for mutable result accumulation such as lists, maps, grouping and joining.

Q122. What is `Collectors.groupingBy()`? ☐☐☐

Extremely useful in real backend coding.

Suppose:

```
java
class Employee {
    String name;
    String department;
}
```

Group employees by department:

```
java
Map<String, List<Employee>> result =
    employees.stream()
        .collect(
            Collectors.groupingBy(
                Employee::getDepartment
            )
        );
```

Result conceptually:

```
text
IT      → [E1, E2]
HR      → [E3]
SALES   → [E4, E5]
```

Real project use case

Suppose your API returns orders.

You need:

```
text
customerId → all orders
```

You can use:

```
java
orders.stream()
    .collect(Collectors.groupingBy(
        Order::getCustomerId
    ));
```

Q123. What is `partitioningBy()`?

It divides elements into two groups based on a predicate.

Example:

```
java
Map<Boolean, List<Integer>> result =
    numbers.stream()
        .collect(
            Collectors.partitioningBy(
                n -> n % 2 == 0
            )
        );
```

Result:

```
text
true  → even numbers
false → odd numbers
```

Difference

`groupingBy()`:

```
text
many possible keys
```

`partitioningBy()`:

```
text
true / false
```

Q124. How do you convert List to Map using Streams? ☐ ☐

Given:

```
java
List<Employee> employees;
```

Convert:

```
text
employeeId → Employee
```

Answer:

```
java
Map<Long, Employee> map =
    employees.stream()
        .collect(
            Collectors.toMap(
                Employee::getId,
                employee -> employee
            )
        );
```

Or:

```
java
Collectors.toMap(
    Employee::getId,
    Function.identity()
)
```

Q125. What happens if duplicate keys occur in `Collectors.toMap()`? ☐

This is a common trap.

Suppose:

```
java
Employee 1 → id 10
Employee 2 → id 10
```

Then:

```
java
Collectors.toMap(
    Employee::getId,
    Function.identity()
)
```

can throw:

```
text
IllegalStateException
```

because the key is duplicated.

Solution

Provide a merge function:

```
java
Map<Long, Employee> map =
    employees.stream()
        .collect(
            Collectors.toMap(
                Employee::getId,
                Function.identity(),
                (oldValue, newValue) -> newValue
            )
        );
```

Now you explicitly decide which duplicate wins.

Q126. What are `skip()` and `limit()`?

limit()

Restricts the number of elements.

```
java
numbers.stream()
    .limit(5)
    .toList();
```

returns at most five.

skip()

Skips the first N elements.

```
java
numbers.stream()
    .skip(5)
    .toList();
```

Real API example

Pagination-like processing:

```
java
stream.skip(offset)
        .limit(pageSize)
```

However, for database-backed applications, don't blindly implement pagination by loading huge datasets into memory and then calling `skip()`. Database pagination should normally be pushed to the database through the repository/query layer.

Q127. What is `distinct()`?

Removes duplicates according to equality semantics.

```
java
List<Integer> result =
    numbers.stream()
            .distinct()
            .toList();
```

Input:

```
text
[1, 2, 2, 3, 3, 3]
```

Output:

```
text
[1, 2, 3]
```

Critical follow-up

For custom objects, correct `equals()` and `hashCode()` are important for meaningful duplicate detection.

Q128. `findFirst()` vs `findAny()`?

`findFirst()`

Returns the first element according to encounter order where applicable.

```
java
Optional<Integer> value =
    numbers.stream()
            .filter(n -> n > 10)
            .findFirst();
```


`findAny()`

Returns any matching element.

This can provide more flexibility for parallel processing.

Interview answer

findFirst() respects encounter order when one exists; findAny() is allowed to return any matching element and can be advantageous in parallel streams.

Q129. What are short-circuiting Stream operations? ☐

Operations that can finish without processing every element.

Examples:

```
text
findFirst()
findAny()
anyMatch()
allMatch()
noneMatch()
limit()
```

Example:

```
java
boolean found =
    numbers.stream()
        .anyMatch(n -> n > 100);
```

Once a matching element is found, processing can stop.

Q130. What is Optional? ☐ ☐

`Optional<T>` represents a value that may or may not be present.

Instead of:

```
java
User user = repository.findById(id);

if (user != null) {
```

```
...  
}
```

Spring Data often uses:

```
java  
Optional<User> user =  
    repository.findById(id);
```

Then:

```
java  
user.ifPresent(...);
```

or:

```
java  
User result =  
    user.orElseThrow(  
        () -> new UserNotFoundException(id)  
    );
```

Purpose

It makes absence explicit in an API.

Q131. `orElse()` vs `orElseGet()` ☐ ☐ ☐

This is a **classic tricky question**.

`orElse()`

The fallback expression is evaluated even when the Optional contains a value.

```
java  
user.orElse(createDefaultUser());
```

`createDefaultUser()` may execute regardless.

`orElseGet()`

The supplier is evaluated only when needed.

```
java  
user.orElseGet(() -> createDefaultUser());
```

Memory trick

```
text
orElse    → eager
orElseGet → lazy
```

This difference matters when the fallback operation is expensive or has side effects.

Q132. What is `orElseThrow()`?

It returns the value if present; otherwise throws an exception.

Example:

```
java
User user =
    repository.findById(id)
        .orElseThrow(
            () -> new UserNotFoundException(id)
        );
```

This is very common in Spring services.

Q133. Should we use Optional as an entity field?

Generally, avoid using `Optional` as a field in JPA entities or DTOs simply because it represents optionality.

Typical use:

```
java
Optional<User> findById(Long id);
```

is much more idiomatic than:

```
java
private Optional<String> name;
```

inside your persistence model.

Interview nuance

`Optional` was primarily designed to improve return-type APIs around potentially absent values, not as a universal replacement for nullable fields.

Q134. What is a parallel stream? □

A parallel stream allows stream processing to be performed using multiple threads.

Example:

```
java
numbers.parallelStream()
    .map(this::process)
    .toList();
```

But don't say:

Parallel stream always makes code faster.

That's incorrect.

Performance depends on:

- dataset size
- CPU workload
- operation characteristics
- splitting efficiency
- shared-state contention
- environment
- ordering requirements

For small collections, parallelism can be slower due to overhead.

Q135. Why can parallel streams be dangerous?

Consider:

```
java
List<Integer> result =
    new ArrayList<>();

numbers.parallelStream()
    .forEach(n -> result.add(n));
```

This is unsafe because `ArrayList` is not thread-safe.

You can get:

- race conditions

- corrupted results
- unexpected behavior

Prefer appropriate collectors:

```
java
List<Integer> result =
    numbers.parallelStream()
        .map(...)
        .toList();
```

and avoid shared mutable state.

Q136. What is the difference between `stream()` and `parallelStream()`?

```
java
list.stream()
```

typically processes sequentially.

```
java
list.parallelStream()
```

allows parallel processing.

Interview answer

I wouldn't choose parallelStream simply because multiple CPU cores exist. I'd benchmark the workload and ensure the operations are safe and suitable for parallel execution.

That's a much stronger product-company answer.

SECTION E — JVM & MEMORY

Q137. What is JVM? ☐ ☐

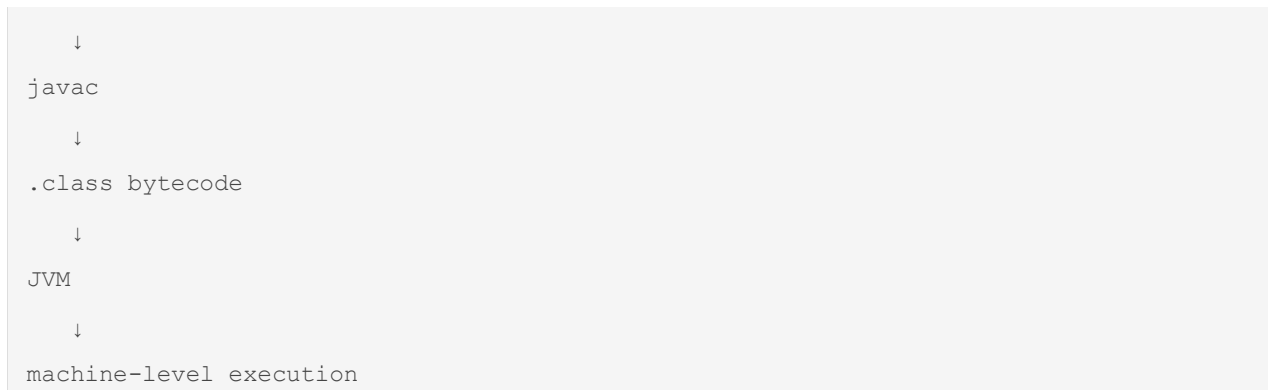
JVM stands for:

Java Virtual Machine.

It executes Java bytecode.

Basic pipeline:

```
text
Java source
```



JVM provides platform independence at the bytecode execution layer.

Q138. JDK vs JRE vs JVM ☐☐

JVM

Executes bytecode.

JRE

Historically:

```
text
JRE = JVM + runtime libraries
```

JDK

Development environment containing tools such as:

```
text
compiler
runtime
debugging/development tools
```

Conceptually:

```
text
JDK
├─ JVM
└─ libraries/tools
```

Modern Java nuance

Since Java 9, Oracle no longer distributes a separate traditional JRE download in the old sense. For interview purposes, understand the conceptual distinction while knowing the modern packaging model.

Q139. What is bytecode?

Java source:

```
java
public class Test {
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

Compiler:

```
text
javac Test.java
```

generates:

```
text
Test.class
```

The `.class` file contains JVM bytecode.

The JVM interprets/JIT-compiles bytecode as appropriate for execution.

Q140. What is JIT compiler? ☐ ☐

JIT stands for:

Just-In-Time compiler.

The JVM can identify frequently executed code and compile it into optimized native machine code at runtime.

Conceptually:

```
text
Bytecode
  ↓
JVM execution
  ↓
hot code detected
  ↓
JIT optimization
```

↓

native machine code

This allows JVM applications to achieve high performance over time.

Q141. What is JVM memory structure? ☐☐☐

At a simplified level:

text

JVM Runtime

Heap
Objects

Thread Stack
Local variables
Method frames

Metaspace
Class metadata

PC Register

Native Method Stack

Important distinction

The heap is shared among threads.

Each thread has its own stack.

Q142. Heap vs Stack ☐☐☐

Stack

Contains execution frames/local state associated with method calls.

Each thread has its own stack.

Example:

```
java
void test() {
    int x = 10;
}
```

The local variable `x` is associated with the current stack frame.

Heap

Objects are allocated in heap memory.

```
java
User user = new User();
```

The `User` object is on the heap.

The reference variable is part of the execution frame/local state.

Strong interview answer

Don't say:

"Primitive variables always live on the stack."

That's an oversimplification.

The exact storage behavior is JVM implementation-dependent and optimized by the JIT.

Q143. What is Metaspace? ☐

Metaspace stores class metadata managed by the JVM.

It replaced the old PermGen approach in Java 8.

Conceptually:

```
text
Java 7 and earlier
```

```
PermGen  
  
Java 8+  
Metaspace
```

Metaspace uses native memory rather than being part of the traditional Java heap.

Q144. What is Garbage Collection? ☐☐☐

Garbage Collection automatically identifies objects that are no longer reachable and reclaims their memory.

Example:

```
java  
User user = new User();  
  
user = null;
```

If no other reachable references point to the object, it may eventually become eligible for GC.

Important

Setting:

```
java  
user = null;
```

does **not** immediately invoke garbage collection.

It only potentially makes the object unreachable.

Q145. Can we force Garbage Collection?

No.

You can request it:

```
java  
System.gc();
```

but this does not guarantee that GC will immediately run.

Interview answer

System.gc() is only a request/hint to the JVM; the JVM is free to decide when and whether to perform garbage collection.

Q146. What is a memory leak in Java?

Java has GC, but memory leaks are still possible.

A leak occurs when objects are no longer logically needed but remain reachable.

Example:

```
java
static List<Object> cache = new ArrayList<>();

public void process(Object obj) {
    cache.add(obj);
}
```

If the list grows indefinitely, objects remain strongly reachable.

GC cannot reclaim them.

Typical causes

- static collections
- unbounded caches
- listeners not removed
- ThreadLocal misuse
- references retained accidentally

Q147. What are strong, weak and soft references?

Strong reference

Normal Java reference:

```
java
User user = new User();
```

As long as the object is strongly reachable, normal GC won't reclaim it.

WeakReference

Allows the referenced object to be collected when no strong references remain.

```
java
WeakReference<User> ref =
    new WeakReference<>(user);
```

SoftReference

Historically intended for memory-sensitive caching where the JVM may reclaim objects under memory pressure.

Interview caution

Don't recommend SoftReference as a general-purpose modern caching strategy. Production caching normally uses explicit cache policies and bounded caches.

Q148. What is ClassLoader? ☐ ☐

A ClassLoader loads classes into the JVM.

Conceptually:

```
text
.class file
  ↓
ClassLoader
  ↓
JVM class representation
```

Common built-in loader categories include:

```
text
Bootstrap ClassLoader
Platform ClassLoader
Application/System ClassLoader
```

Why important?

Class loading affects:

- dependency resolution
- class identity
- application startup
- frameworks
- plugin architectures

Q149. What is the parent delegation model? ☐ ☐

When a class loader receives a request to load a class, it generally delegates to its parent first.

Conceptually:

```
text
Application ClassLoader
    ↓
Platform ClassLoader
    ↓
Bootstrap ClassLoader
```

The parent tries first.

Only if the parent cannot load the class does the child generally attempt to load it.

Why?

It helps:

- prevent application classes from replacing core Java classes
- maintain class-loading consistency
- improve security

Q150. What is Reflection? ☐ ☐

Reflection allows a Java program to inspect and interact with classes, methods, fields, constructors, etc., at runtime.

Example:

```
java
Class<?> clazz = User.class;

System.out.println(clazz.getName());

Method[] methods =
    clazz.getDeclaredMethods();
```

Frameworks heavily use reflection or related runtime mechanisms.

Examples include:

text

Spring
Hibernate
JUnit
Jackson

Spring example

When Spring sees:

java

```
@Service  
public class PaymentService {  
}
```

Spring's infrastructure analyzes class metadata and manages the resulting bean.

Tricky follow-up

Why not use reflection everywhere?

Because reflection can:

- reduce type safety
- complicate code
- introduce runtime failures
- have overhead
- make refactoring harder

Framework infrastructure is where it is particularly useful.

☐ MODULE 1 COMPLETE — 150/150

You now have the complete Core Java module:

text

MODULE 1

Q1-50
Core Java
OOP
Strings

Exceptions
Object class
Memory fundamentals

Q51-100
Collections
ArrayList
LinkedList
HashMap
HashSet
TreeMap
LinkedHashMap
ConcurrentHashMap
Iterator
Generics
Type Erasure

Q101-150
Functional Interfaces
Lambdas
Streams
map / flatMap
reduce
Collectors
Optional
Parallel Streams
JVM
JIT
Heap / Stack
GC
Metaspace
ClassLoader
Reflection

☐ TOP 30 FROM MODULE 1 — MUST BE PERFECT

If you have limited preparation time, prioritize these:

#	Topic
1	<code>==</code> vs <code>equals()</code>
2	<code>equals()</code> + <code>hashCode()</code> contract
3	String immutability
4	String pool
5	final/finally/finalize
6	ArrayList vs LinkedList
7	HashMap internal working
8	HashMap collision
9	HashMap load factor
10	HashMap resizing
11	HashMap mutable keys
12	HashMap vs ConcurrentHashMap
13	HashSet internal working
14	Comparable vs Comparator
15	Iterator / ConcurrentModificationException
16	Fail-fast
17	Generics
18	Type erasure
19	Functional interface
20	Lambda
21	Predicate/Function/Consumer/Supplier
22	<code>map()</code> vs <code>flatMap()</code>
23	<code>reduce()</code>
24	<code>groupingBy()</code>
25	<code>toMap()</code> duplicate keys
26	<code>orElse()</code> vs <code>orElseGet()</code>
27	Stream lazy evaluation

#	Topic
28	Parallel Stream
29	Heap vs Stack
30	JVM/JIT/GC

These topics line up particularly well with recent 2-year Java interview guidance and actual recent interview reports, where candidates have been asked HashMap internals, Streams, `equals()`, concurrency, `CompletableFuture`, and practical Spring Boot questions in the same process. ([CodeBegun](#))

★ Next Module: Spring & Spring Boot — 120 Questions

Now we move to the part that is **most important for your stated target: 2-year Spring Boot backend/product-company interviews**.

I will **not** make Module 2 merely:

What is @Autowired? What is @Component?

Instead, we'll build it like an actual product-company interview.

Module 2 structure

Part A — Spring Core

Q1–30

- What is Spring?
- IoC
- DI
- BeanFactory
- ApplicationContext
- Bean lifecycle
- Constructor injection
- `@Autowired`
- `@Component`
- `@Service`
- `@Repository`
- `@Controller`
- `@Configuration`
- `@Bean`
- singleton/prototype/request/session scopes

- circular dependencies
- lazy initialization
- `@Primary`
- `@Qualifier`
- Spring proxies
- AOP

Part B — Spring Boot Internals

Q31–60

- Spring vs Spring Boot
- `@SpringBootApplication`
- auto-configuration
- starters
- conditional configuration
- `@ConditionalOnClass`
- `@ConditionalOnMissingBean`
- configuration properties
- profiles
- application startup
- embedded server
- Actuator
- custom configuration
- overriding Boot defaults
- debugging auto-configuration

Current 2026 interview material specifically emphasizes dependency injection, IoC, auto-configuration, starters, configuration/profiles, and understanding what happens internally when dependencies are added. ([CodeBegun](#))

Part C — Spring MVC + REST

Q61–85

- DispatcherServlet
- Controller → Service → Repository
- `@RequestMapping`
- `@GetMapping`
- `@PostMapping`
- `@PathVariable`

- `@RequestParam`
- `@RequestBody`
- DTOs
- validation
- `@Valid`
- global exception handling
- `@ControllerAdvice`
- `ResponseEntity`
- HTTP status codes
- idempotency
- pagination
- API versioning

Part D — Production Spring Boot

Q86–105

- Transactions
- `@Transactional`
- propagation
- isolation
- rollback
- proxy behavior
- self-invocation
- transaction boundaries
- N+1
- connection pools
- caching
- logging
- Actuator
- health checks
- configuration/security mistakes

Part E — Actual Interview Scenarios

Q106–120

These will be the **most valuable**.

For example:

"Your API is suddenly taking 5 seconds instead of 200 ms. How do you investigate?"

"You added @Transactional, but rollback isn't happening. Why?"

"Two beans implement the same interface. What happens?"

"Your application starts locally but fails in production. How do you debug it?"

"Why does adding spring-boot-starter-data-jpa suddenly create database-related infrastructure?"

"Your API returns 500 for invalid input. How would you redesign the exception handling?"

"Your Spring Boot service has 10,000 DB queries for one API request. What's happening?"

These are the questions that turn **memorization into interview performance**.

Recent 2-year interview guidance specifically recommends being able to build a small end-to-end feature—**controller** → **service** → **repository** → **DTO** → **validation** → **exception handling**—and explain the design decisions, rather than only knowing annotation definitions. ([CodeBegun](#))

We will therefore start **Module 2, Q1–30: Spring Core + IoC/DI internals** next.